



Day 2 — Arrays, Traversal & Core Problem-Solving Patterns

তোমার বর্তমান অবস্থান অনুযায়ী Day 2 হবে **খুব গুরুত্বপূর্ণ turning point**।

Day 0-তে তুমি শিখেছো **CP কীভাবে ভাবতে হয়**।

Day 1-এ তুমি শিখেছো **Problem পড়া → Pattern চিন্তা → Algorithm → Dry Run → Code → Test workflow**।

এখন Day 2 থেকে আমরা সত্যিকারের **problem-solving pattern training** শুরু করব।

আজকের মূল লক্ষ্য:

Array Problem

↓

একটা একটা Element দেখা

↓

Condition Check করা

↓

Result জমা / Count / Compare / Search করা

Day 2 শেষে তুমি যেন এই ধরনের Problem নিজে চিনতে পারো:

Sum বের করো

Maximum বের করো

Minimum বের করো

Even সংখ্যা Count করো

Target Search করো

Positive / Negative Count করো



Day 2 Chapter Map

Day 2

- Chapter 1 – Array Mental Model
- Chapter 2 – Traversal Pattern
- Chapter 3 – Accumulation Pattern
- Chapter 4 – Counting Pattern
- Chapter 5 – Maximum & Minimum Pattern
- Chapter 6 – Searching Pattern
- Chapter 7 – Combining Patterns
- Chapter 8 – Common Bugs & Edge Cases
- Chapter 9 – Problem Solving Session
- Chapter 10 – Pattern Library Update
- Chapter 11 – Assignment & Reflection

আমি Day 1-এর মতোই **বইয়ের Chapter style**-এ এগোব।

আজ শুরু করছি **Day 2 — Chapter 1** দিয়ে।

Day 2 — Chapter 1

Array Mental Model

Lesson 1 — Array গুণ্ডু Syntax না

তুমি ইতোমধ্যে C-তে Array জানো।

যেমন:

```
int arr[5];
```

অথবা:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Programming class-এ সাধারণত শেখানো হয়:

Array হলো একই Data Type-এর একাধিক Value রাখার একটি Data Structure।

Definition হিসেবে এটা ঠিক।

কিন্তু CP-এর জন্য আমাদের আরেকভাবে ভাবতে হবে।

Array হলো অনেকগুলো Data, যেগুলোর উপর আমাদের কোনো Operation করতে হবে।

যেমন:

```
10 20 30 40 50
```

Problem বলতে পারে:

Sum বের করো।

অথবা:

Maximum বের করো।

অথবা:

Even সংখ্যা Count করো।

অথবা:

৩০ আছে কি না খুঁজে বের করো।

লক্ষ্য করো—

Array একই।

শুধু Operation বদলে যাচ্ছে।

Lesson 2 — Array Problem-এর Basic Mental Model

যখন Array দেখবে, প্রথম চিন্তা হবে না:

for loop লিখি।

বরং চিন্তা হবে:

আমাকে প্রতিটি Element দেখতে হবে?

যদি উত্তর হয়:

হ্যাঁ

তাহলে:

Need Traversal

এরপর প্রশ্ন:

প্রতিটি Element দেখে কী করব?

Possible Answer:

যোগ করব

তাহলে:

Traversal

↓

Accumulation

যদি বলো:

Condition Check করে Count করব

তাহলে:

Traversal

↓

Condition

↓

Counter

যদি বলো:

বর্তমান Maximum-এর সঙ্গে Compare করব

তাহলে:

Traversal

↓

Comparison

↓

Maximum Update

এটাই আজকের Day 2-এর foundation।

Lesson 3 — Index এবং Element এক জিনিস না

ধরো:

Index: 0 1 2 3 4

Element: 10 20 30 40 50

এখানে:

i

হলো Index।

আর:

arr[i]

হলো সেই Index-এর Value বা Element।

উদাহরণ:

i = 2

তাহলে:

arr[i]

মানে:

arr[2]

Value:

30

অর্থাৎ:

`i` → Position / Index

`arr[i]` → Value at that position

এই পার্থক্য পরিষ্কার থাকা খুব জরুরি।

Common Beginner Mistake

ধরো Even Number Print করতে হবে।

ভুল:

```
if (i % 2 == 0)
{
    printf("%d ", arr[i]);
}
```

এখানে তুমি check করছো:

Index Even কি না

কিন্তু Problem যদি বলে:

Even Value Print করো

তাহলে হবে:

```
if (arr[i] % 2 == 0)
{
    printf("%d ", arr[i]);
}
```

কারণ:

`i % 2`

মানে:

Index Check

আর:

`arr[i] % 2`

মানে:

Value Check

Lesson 4 — Traversal কী?

Traversal তুমি Glossary-তে ইতোমধ্যে দেখেছো।

সহজভাবে:

Collection-এর Element-গুলো একে একে Visit করাকে Traversal বলে।

Array:

5 8 2 9 1

Traversal:

5
↓
8
↓
2
↓
9
↓
1

Code:

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}
```

কিন্তু এখানে একটা গুরুত্বপূর্ণ বিষয় আছে।

Loop এবং Traversal একই জিনিস নয়।

Loop হলো Mechanism।

Traversal হলো Purpose।

for loop

↓

Tool

Traversal

↓

Operation / Process

উদাহরণ:

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}
```

এখানে:

Loop



Array Traversal করছে

Lesson 5 — Traversal-এর তিনটি প্রশ্ন

যেকোনো Array Problem-এ Traversal করার আগে নিজেকে তিনটা প্রশ্ন করবে।

Question 1

কোথা থেকে শুরু করব?

সাধারণত:

$i = 0$

Question 2

কতক্ষণ চলব?

সাধারণত:

$i < n$

Question 3

প্রতিটি Element-এর সঙ্গে কী করব?

এটাই Problem অনুযায়ী বদলাবে।

Sum

Add

Count Even

Check

↓

Count

Maximum

Compare

↓

Update

Search

Compare with Target

↓

Found?

Lesson 6 — Same Traversal, Different Problems

এই Lesson খুব গুরুত্বপূর্ণ।

ধরো Array:

5 8 2 9 4

একই Traversal:

```
for (int i = 0; i < n; i++)  
{  
    // কাজ  
}
```

এখন Problem অনুযায়ী ভিতরের কাজ বদলাবে।

Problem A — Print All Elements

```
printf("%d ", arr[i]);
```

Mental Model:

Traversal

↓

Output

Problem B — Sum

```
sum += arr[i];
```

Mental Model:

Traversal

↓

Accumulation

Problem C — Count Even

```
if (arr[i] % 2 == 0)
{
    count++;
}
```

Mental Model:

Traversal
↓
Condition
↓
Counter

Problem D — Maximum

```
if (arr[i] > max)
{
    max = arr[i];
}
```

Mental Model:

Traversal
↓
Comparison
↓
Update

Problem E — Search

```
if (arr[i] == target)
{
    found = 1;
}
```

Mental Model:

```
Traversal
  ↓
Target Comparison
  ↓
State Update
```

এই কারণেই আমি তোমাকে বলেছিলাম:

Pattern-কে শুধু একটি শব্দ হিসেবে দেখবে না।

যেমন শুধু:

```
Maximum
```

না লিখে ভাববে:

```
Traversal
  ↓
Comparison
  ↓
Update Maximum
```

Lesson 7 — The Core Array Skeleton

অনেক Beginner Array Problem-এর ভিতরের common structure দেখতে পায় না।

দেখো:

```
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Problem Logic

    return 0;
}
```

এই পর্যন্ত অংশ অনেক Array Problem-এ common।

তারপর Problem Logic বদলাবে।

Sum Problem

```
int sum = 0;

for (int i = 0; i < n; i++)
{
    sum += arr[i];
}
```

Count Problem

```
int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] % 2 == 0)
    {
        count++;
    }
}
```

Maximum Problem

```
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

লক্ষ্য করো:

```
Input
  ↓
Traversal
  ↓
Operation
  ↓
Result
```

এই skeleton মাথায় ঢোকানো Day 2-এর বড় লক্ষ্য।

Lesson 8 — Read and Process: সব সময় Array লাগবে?

না।

এটা খুব গুরুত্বপূর্ণ।

ধরো Problem:

Nটি Number Input নিয়ে Sum বের করো।

তুমি লিখতে পারো:

```
int arr[n];
```

সব Number Array-তে রাখতে পারো।

কিন্তু Sum-এর জন্য সব Value পরে আবার দরকার নেই।

তাই সরাসরি:

```
int sum = 0;
int x;

for (int i = 0; i < n; i++)
{
    scanf("%d", &x);
    sum += x;
}
```

করাও সম্ভব।

Mental Model:

```
Read
↓
Process
↓
Forget
```

অন্যদিকে যদি পরে Array-এর Values আবার দরকার হয়:

Read

↓

Store

↓

Process Later

তাহলে Array লাগবে।

Lesson 9 — Store or Process?

নিজেকে প্রশ্ন করবে:

এই Value পরে আবার দরকার হবে?

যদি:

না

তাহলে:

Read

↓

Process Immediately

যদি:

হ্যাঁ

তাহলে:

Read

↓

Store in Array

↓

Process

Example: Sum

শুধু Sum দরকার:

10

20

30

Processing:

sum = 0

Read 10

sum = 10

Read 20

sum = 30

Read 30

sum = 60

Array না রাখলেও সম্ভব।

Example: Reverse Array

Input:

10 20 30 40

Output:

40 30 20 10

এখানে পরে আগের Value-গুলো দরকার।

তাই:

Need Storage

↓

Array

Lesson 10 — Array Problem Translation

Problem Statement:

Given N integers, find the number of positive integers.

Beginner Brain:

Code কী হবে?

আমাদের Brain:

N Numbers

↓

Need to inspect each number

↓

Traversal

Need positive numbers only

↓

Condition: $x > 0$

Need how many

↓

Counter

Final Thinking Pipeline:

Traversal



Condition Checking



Counter

তারপর Code।

আরেকটা:

Find the smallest number in an array.

Thinking:

Need every Element



Traversal

Need Smallest



Comparison

Found Smaller?



Update Minimum

Pipeline:

Traversal



Comparison

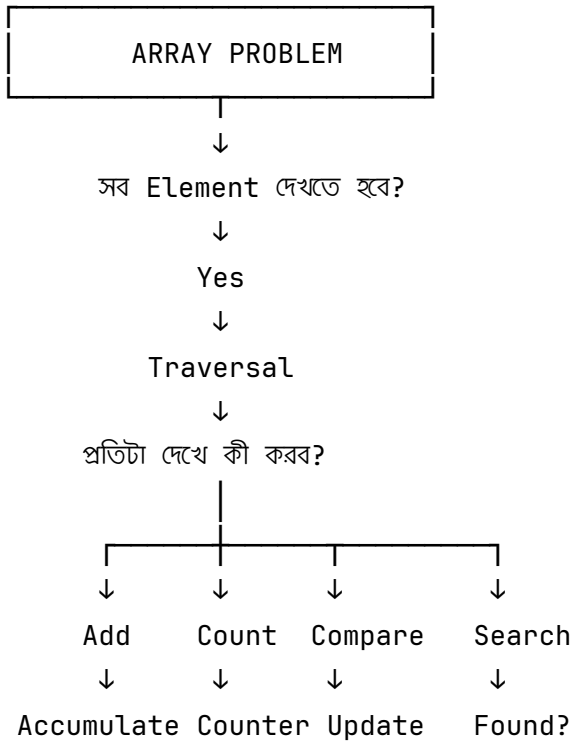


Minimum Update



Chapter 1 Core Mental Model

আজ থেকে Array Problem দেখলে:



এই Diagram Day 2-এর পুরো foundation।



Chapter 1 — Exercise

এখন Code লিখবে না।

শুধু **Thinking Pipeline** লিখবে।

Exercise 1

Given N integers, find their sum.

Format:

Pattern:

???

↓

???

Exercise 2

Count how many numbers are negative.

Pattern:

???

↓

???

↓

???

Exercise 3

Find the largest number.

Pattern:

???

↓

???

↓

???

Exercise 4

Find whether the number 7 exists in an array.

Pattern:

???

↓

???

↓

???

Exercise 5

Print all odd numbers.

Pattern:

???

↓

???

↓

???

Exercise 6 — Important

Problem:

Given N numbers, calculate their sum.

Question:

Array কি অবশ্যই লাগবে?

Yes / No

কারণ:

Exercise 7

Problem:

Reverse N numbers.

Question:

Array কেন দরকার হতে পারে?

কারণ:



Chapter 1 Assignment

তোমার 02_Day_2/Assignment_and_Exercise/ folder-এ:

Day_2_chapter_1_exercise.md

বানাতে পারো।

Structure:

Day 2 – Chapter 1 Exercise

Exercise 1

Pattern

Traversal

↓

...

Why?

...

Exercise 2

Pattern

...

Why?

...

এবার শুধু pattern-এর নাম দিলেই হবে না। প্রতিটায় **Why?** লিখবে।

যেমন:

Traversal

↓

Condition

↓

Counter

তারপর:

Why?

প্রতিটি Number দেখতে হবে, Negative কি না Check করতে হবে,
Negative হলে Counter এক বাড়তে হবে।

এতে আমি বুঝতে পারব তুমি শব্দ মুখস্থ করেছো নাকি logic বুঝেছো।

Day 2 — Chapter 2



Traversal Pattern



Chapter Goal

এই Chapter শেষে তুমি বুঝতে পারবে:

- Traversal আসলে কী;
- Loop আর Traversal-এর পার্থক্য;
- Forward Traversal কী;
- Reverse Traversal কী;
- Full Traversal এবং Partial Traversal কী;
- কখন Traversal দরকার;
- `i` এবং `arr[i]` -এর পার্থক্য;
- Common Traversal Bug কী;
- Problem পড়ে কীভাবে Need Traversal চিনতে হয়।

এই Chapter-এর মূল লক্ষ্য:

Problem Statement

↓

Need to inspect elements?

↓

Traversal

↓

Choose Direction

↓

Perform Operation

Lesson 1 — Traversal আবার পরিষ্কার করি

Traversal মানে:

কোনো Collection-এর Element-গুলো একটি নির্দিষ্ট Order-এ Visit করা।

ধরো Array:

10 20 30 40 50

Forward Traversal:

10

↓

20

↓

30

↓

40

↓

50

Code:

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}
```

এখানে Loop ব্যবহার করে Array Traversal করা হচ্ছে।

Lesson 2 — Loop এবং Traversal এক জিনিস নয়

এই পার্থক্যটা খুব গুরুত্বপূর্ণ।

Loop

Loop হলো একটি **Control Structure**।

এটা কোনো কাজ বারবার করতে সাহায্য করে।

Example:

```
for (int i = 1; i ≤ 5; i++)  
{  
    printf("Hello\n");  
}
```

এখানে Loop আছে।

কিন্তু কোনো Array বা Collection Visit করা হচ্ছে না।

তাই এটাকে Array Traversal বলা হবে না।

Traversal

Traversal হলো:

Data-এর Element-গুলো Visit করার Process।

Example:

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}
```

এখানে:

for loop

↓

Mechanism / Tool

Array-এর সব Element Visit

↓

Traversal

মনে রাখার Rule

Loop = কীভাবে বারবার করছি?

Traversal = কোন Data-গুলো একে একে দেখছি?

Lesson 3 — Iteration এবং Traversal

আরেকটা নতুন শব্দ:

Iteration

Loop-এর প্রতিবার Execution-কে একটি **Iteration** বলা হয়।

ধরো:

```
for (int i = 0; i < 5; i++)  
{  
    printf("%d\n", i);  
}
```

এখানে Loop চলবে ৫ বার।

```
i = 0 → Iteration 1  
i = 1 → Iteration 2  
i = 2 → Iteration 3  
i = 3 → Iteration 4  
i = 4 → Iteration 5
```

অর্থাৎ:

Traversal

↓

অনেকগুলো Iteration দিয়ে সম্পন্ন হতে পারে

Lesson 4 — Forward Traversal

সবচেয়ে common Traversal:

Left → Right

Array:

Index:	0	1	2	3	4
--------	---	---	---	---	---

Value:	10	20	30	40	50
--------	----	----	----	----	----

Traversal:

arr[0]
↓
arr[1]
↓
arr[2]
↓
arr[3]
↓
arr[4]

Code:

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}
```

Dry Run

ধরো:

n = 4

arr = 5 8 2 9

Dry Run:

i	arr[i]	Output
0	5	5
1	8	8
2	2	2
3	9	9

Final Output:

5 8 2 9

Lesson 5 — Reverse Traversal

সব সময় Left থেকে Right যেতে হবে না।

Array:

10 20 30 40 50

Reverse Traversal:

50
↓
40
↓
30
↓
20
↓
10

Index:

4 → 3 → 2 → 1 → 0

Code:

```
for (int i = n - 1; i ≥ 0; i--)  
{  
    printf("%d ", arr[i]);  
}
```

কেন $n - 1$?

ধরো:

$$n = 5$$

Valid Index:

0 1 2 3 4

শেষ Index:

4

আর:

$$n - 1$$

=

$$5 - 1$$

=

4

তাই Reverse Traversal শুরু হয়:

```
int i = n - 1;
```

থেকে।

Lesson 6 — Traversal Direction কীভাবে ঠিক করব?

নিজেকে প্রশ্ন করবে:

কোন Order-এ Elementগুলো Process করতে হবে?

যদি Problem বলে:

Print all elements.

সাধারণত:

Forward Traversal

যদি Problem বলে:

Print elements in reverse order.

তাহলে:

Reverse Traversal

যদি Problem বলে:

Find Maximum.

তাহলে সাধারণত:

Forward Traversal

যদিও Maximum বের করতে technically অন্য direction-ও সম্ভব, standard approach হলো forward traversal।

Lesson 7 — Full Traversal

যখন সব Element Visit করা হয়:

10 20 30 40 50

↑ ↑ ↑ ↑ ↑

সবগুলো Visit

এটা:

Full Traversal

Code:

```
for (int i = 0; i < n; i++)  
{  
    // process arr[i]  
}
```

Common Problems:

- Sum
- Count
- Maximum
- Minimum
- Average
- Frequency

এগুলোতে সাধারণত Full Traversal লাগে।

Lesson 8 — Partial Traversal

সব Problem-এ শেষ পর্যন্ত যাওয়া দরকার হয় না।

ধরো:

Array-তে 7 আছে কি না Check করো।

Array:

2 5 7 9 10

Traversal:

2 → Not Found

5 → Not Found

7 → Found

এখন যদি Problem শুধু জানতে চায়:

7 আছে কি না?

তাহলে 7 পাওয়ার পর:

`break;`

করা যায়।

Example:

```
for (int i = 0; i < n; i++)  
{  
    if (arr[i] == 7)  
    {  
        found = 1;  
        break;  
    }  
}
```

Mental Model:

Traversal

↓

Target Comparison

↓

Found?

↓

Stop Early

এটাকে **Early Termination** বা **Early Exit** বলা যায়।

Lesson 9 — Traversal + Operation

Traversal নিজে সাধারণত Final Solution না।

Traversal হলো Data দেখার Process।

তারপর প্রতিটি Element-এর উপর কোনো Operation হয়।

Example 1 — Print

Traversal

↓

Output

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}
```

Example 2 — Sum

Traversal

↓

Accumulation

```
for (int i = 0; i < n; i++)  
{  
    sum += arr[i];  
}
```

Example 3 — Count

Traversal

↓

Condition

↓

Counter

```
for (int i = 0; i < n; i++)
{
    if (arr[i] > 0)
    {
        count++;
    }
}
```

Example 4 — Maximum

Traversal

↓

Comparison

↓

Update

```
for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

Lesson 10 — Index Traversal vs Value Processing

এটা Beginner-দের খুব common confusion।

ধরো:

Index:	0	1	2	3
--------	---	---	---	---

Value:	7	12	5	8
--------	---	----	---	---

Loop:

```
for (int i = 0; i < n; i++)
```

এখানে:

i

হবে:

0

1

2

3

কিন্তু:

arr[i]

হবে:

7

12

5

8

তাই:

```
if (i % 2 == 0)
```

মানে:

Even Index Check

আর:

```
if (arr[i] % 2 == 0)
```

মানে:

Even Value Check

Example

Problem:

Print all even values.

Correct:

```
for (int i = 0; i < n; i++)  
{  
    if (arr[i] % 2 == 0)  
    {  
        printf("%d ", arr[i]);  
    }  
}
```

Problem:

Print values stored at even indices.

Correct:

```
for (int i = 0; i < n; i++)
{
    if (i % 2 == 0)
    {
        printf("%d ", arr[i]);
    }
}
```

দুটো Problem সম্পূর্ণ আলাদা।

Lesson 11 — Off-by-One Error

Traversal-এর সবচেয়ে common Bug-এর একটি হলো:

একটি Element বেশি বা কম Process করা।

এটাকে সাধারণভাবে **Off-by-One Error** বলা হয়।

ধরো:

n = 5

Valid Index:

0 1 2 3 4

Correct:

```
for (int i = 0; i < n; i++)
```

অর্থাৎ:

0 1 2 3 4

Wrong Version 1

```
for (int i = 0; i ≤ n; i++)
```

এখানে i যাবে:

0 1 2 3 4 5

কিন্তু:

arr[5]

Valid না।

Wrong Version 2

```
for (int i = 0; i < n - 1; i++)
```

এখানে:

0 1 2 3

Visit হবে।

শেষ Element:

arr[4]

বাদ পড়বে।

Lesson 12 — Empty Loop Body Bug

এই Bug-টা C-তে খুব dangerous।

ভুল:

```
for (int i = 0; i < n; i++);  
{  
    printf("%d ", arr[i]);  
}
```

খেয়াল করো:

```
for (...);
```

শেষে accidental semicolon আছে।

এতে Loop-এর body empty হয়ে যায়।

Correct:

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", arr[i]);  
}
```

Contest-এ ছোট syntax mistake অনেক সময় বেশি সময় নষ্ট করায়। তাই Loop header দেখার habit তৈরি করো।

Lesson 13 — Traversal Recognition Signals

Problem Statement-এ এই ধরনের কথা দেখলে Traversal চিন্তা করবে:

For each element...

Among N numbers...

Count how many...

Find the largest...

Find the smallest...

Print all...

Check whether X exists...

তখন Brain:

Multiple Elements

↓

Need to inspect them

↓

Traversal

কিন্তু এরপর থামবে না।

পরের প্রশ্ন:

Traversal করে কী করব?

Possible answer:

Print

Add

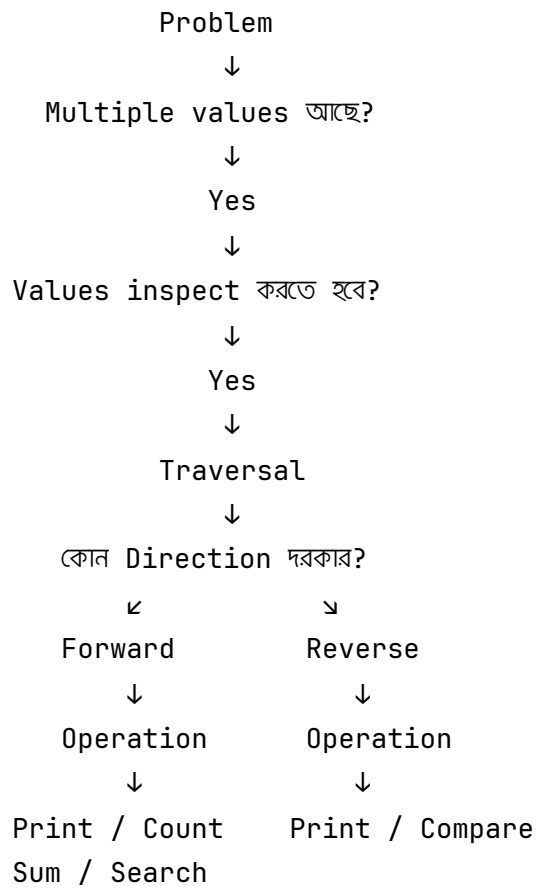
Count

Compare

Search



Traversal Decision Map



Worked Example 1

Problem:

Given N integers, print all positive numbers.

Input:

6
-2 5 0 -7 9 3

Step 1 — Need every Element?

Yes

তাই:

Traversal

Step 2 — কী Check করব?

Positive?

Condition:

```
arr[i] > 0
```

Step 3 — কী করব?

Print

Final Pipeline:

Traversal

↓

Condition Checking

↓

Output

Code:

```
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    for (int i = 0; i < n; i++)
    {
        if (arr[i] > 0)
        {
            printf("%d ", arr[i]);
        }
    }

    return 0;
}
```



Worked Example 2

Problem:

Print an array in reverse order.

Input:

```
5
10 20 30 40 50
```

Need:

Last Element

↓

Previous Element

↓

...

↓

First Element

Pipeline:

Reverse Traversal

↓

Output

Code:

```
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    for (int i = n - 1; i ≥ 0; i--)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

⚠ Important: Reverse Printing vs Reversing an Array

দুটো সব সময় একই কাজ না।

এই Code:

```
for (int i = n - 1; i ≥ 0; i--)  
{  
    printf("%d ", arr[i]);  
}
```

শুধু Reverse Order-এ **Print** করছে।

Original Array-এর ভিতরের order বদলাচ্ছে না।

অর্থাৎ:

Original Array:

10 20 30 40 50

Reverse Print:

50 40 30 20 10

কিন্তু memory-তে Array এখনও:

10 20 30 40 50

Actual Array Reverse করতে হলে Element swap করতে হবে। সেটা পরে appropriate pattern-এর সঙ্গে করব।



Chapter 2 Exercise

এখানে Code লিখবে। তবে ছোট Problem—আজকের একদিনের pacing বজায় রাখার জন্য শুধু ৪টা exercise।

Exercise 1 — Forward Traversal

Input:

```
5
10 20 30 40 50
```

Output:

```
10 20 30 40 50
```

Required Pattern

```
Traversal
↓
Output
```

Exercise 2 — Reverse Traversal

Input:

```
5
10 20 30 40 50
```

Output:

```
50 40 30 20 10
```

Required Pattern

Reverse Traversal

↓

Output

Exercise 3 — Print Even Values

Input:

6
3 8 5 12 7 10

Output:

8 12 10

Required Pattern

নিজে লিখবে।

Hint:

Traversal

↓

???

↓

Output

Exercise 4 — Print Values at Even Indices

Input:

6

10 20 30 40 50 60

Index:

0 1 2 3 4 5

Output:

10 30 50

এখানে খেয়াল রাখবে:

Even Value

এবং:

Even Index

এক না।



Chapter 2 Assignment

Code না লিখে প্রথমে Thinking Pipeline লিখবে।

Problem 1

Print all negative numbers.

লিখবে:

Pattern:

???

↓

???

↓

???

Problem 2

Print an array from last to first.

Pattern:

???

↓

???

Problem 3

Check every element and print only values greater than 100.

Pattern:

???

↓

???

↓

???

Problem 4

Search for `x` and stop immediately after finding it.

এখানে নতুন concept:

Early Termination

নিজে পুরো Pipeline বানাবে।



তোমার Repo-তে কোথায় রাখবে?

Day 2-এর learning material:

```
02_Day_2/  
├── day_2.md  
└── Assignment_and_Exercise/  
    ├── Day_2_exercise.md  
    └── Day_2_assignment.md
```

আজকের local exercise code চাইলে:

```
02_Day_2/  
└── Practice/  
    ├── traversal_forward.c  
    ├── traversal_reverse.c  
    ├── even_values.c  
    └── even_indices.c
```

কিন্তু [HackerRank](#)-এর actual problem solution হলে সেটা Day folder-এ duplicate করবে না; তোমার existing platform folder `19_HackerRank/` -এর relevant category-তে রাখবে।



Day 2 Pacing Correction

তোমার কথাটা গুরুত্বপূর্ণ: **Day 2 একদিনেই শেষ হতে হবে এবং পুরো Bootcamp ১৬ দিনের মধ্যেই শেষ হতে হবে।**

তাই Day 2-এর বাকি অংশ আমি unnecessarily বড় করব না। Revised practical flow হবে:

Chapter 1

Array Mental Model



Chapter 2

Traversal



Chapter 3

Accumulation + Counting



Chapter 4

Maximum + Minimum



Chapter 5

Linear Search



Chapter 6

Mixed Pattern Problems



Chapter 7

Mistake Review + Pattern Library Update



Day 2 Final Assignment

অর্থাৎ আগের ১১ Chapter plan compress করে **৭টি focused Chapter** করলাম। Concept বাদ যাচ্ছে না; related concepts একসঙ্গে group করছি।

আজকের Day 2-এর target হবে:

Learn



Traversal

Accumulation

Counting

Maximum

Minimum

Linear Search



Solve Mixed Problems



Review Mistakes



Day 2 Complete

Chapter 2 এখানেই শেষ। Exerciseগুলো করার পর Chapter 3-এ আমরা **Accumulation + Counting Pattern** একসঙ্গে শিখব—কারণ ১৬ দিনের bootcamp pacing অনুযায়ী related patterns আলাদা আলাদা বিশাল Chapter করলে practical preparation পিছিয়ে যাবে।

Day 2 — Chapter 3



Accumulation + Counting Pattern

Chapter 2-এ আমরা শিখেছি:

Multiple Elements



একটা একটা করে Visit



Traversal

কিন্তু শুধু Element Visit করলেই সাধারণত Problem Solve হয় না।

প্রতিটি Element দেখে আমাদের কিছু একটা করতে হয়।

আজ আমরা দুইটা খুব গুরুত্বপূর্ণ Pattern শিখব:

1. Accumulation
2. Counting

দুটো দেখতে কাছাকাছি হলেও কাজ আলাদা।

এই Chapter শেষে তুমি যেন Problem পড়ে বুঝতে পারো:

Total / Sum দরকার?



Accumulator

How Many / Count দরকার?



Counter

Chapter Goal

এই Chapter শেষে তুমি বুঝতে পারবে:

- Accumulator কী;
- Counter কী;
- Accumulator এবং Counter-এর পার্থক্য;
- Running Sum কী;
- Initialization কেন গুরুত্বপূর্ণ;
- `sum += arr[i]` কীভাবে কাজ করে;
- `count++` কখন ব্যবহার করতে হয়;
- Condition + Counter Pattern;
- Sum এবং Count combine করে Average বের করা;
- কোন Problem-এ Array না রেখেও কাজ করা যায়;
- Common accumulation এবং counting bugs।

আজকের Core Mental Model:

Problem



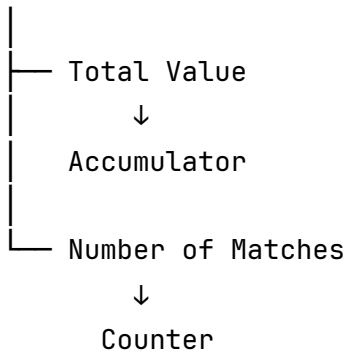
সব Value দেখতে হবে?



Traversal



কী Result দরকার?



Lesson 1 — Accumulation Pattern কী?

ধরো Array:

2 5 3 10

Problem:

সব Number-এর Sum বের করো।

আমাদের সব Number একসঙ্গে যোগ করতে হবে।

প্রথমে:

sum = 0

তারপর:

Read 2

$\text{sum} = 0 + 2$

$\text{sum} = 2$

তারপর:

Read 5

$\text{sum} = 2 + 5$

$\text{sum} = 7$

তারপর:

Read 3

$\text{sum} = 7 + 3$

$\text{sum} = 10$

তারপর:

Read 10

$\text{sum} = 10 + 10$

$\text{sum} = 20$

Final:

$\text{sum} = 20$

এখানে `sum` Variable প্রতিবার Result জমা করছে।

এই ধরনের Variable-কে বলা হয়:

Accumulator

Mental Model:

Traversal



Current Value নাও



Previous Result-এর সঙ্গে যোগ করো



Accumulator Update করো

Lesson 2 — Accumulator শব্দটার মানে

সহজ বাংলায়:

যে Variable ধাপে ধাপে Result জমা করে, তাকে Accumulator বলা হয়।

Example:

```
int sum = 0;
```

তারপর:

```
sum = sum + arr[i];
```

অথবা Short Form:

```
sum += arr[i];
```

দুটো একই কাজ করে।

Example

ধরো:

```
arr[i] = 5
```

এবং:

```
sum = 10
```

তাহলে:

```
sum += arr[i];
```

মানে:

```
sum = sum + arr[i]
```

```
sum = 10 + 5
```

```
sum = 15
```

এখন Accumulator-এর নতুন State:

```
sum = 15
```

Lesson 3 — Running Sum

Accumulation চলার সময় প্রতিটি ধাপের Sum-কে আমরা **Running Sum** বলতে পারি।

Array:

```
4 2 7 3
```

Dry Run:

Current Value	Previous Sum	New Sum
4	0	4
2	4	6
7	6	13

Current Value	Previous Sum	New Sum
3	13	16

এখানে:

0
↓
4
↓
6
↓
13
↓
16

এইভাবে Result ধাপে ধাপে তৈরি হচ্ছে।

Final Answer:

16

Lesson 4 — Sum Problem-এর Complete Thinking

Problem:

Given N integers, find their sum.

Problem পড়েই Code লিখবে না।

প্রথমে Translation:

Step 1 — কী দরকার?

সব Number-এর Total

Step 2 — সব Number দেখতে হবে?

Yes

তাই:

Traversal

Step 3 — প্রতিটি Number নিয়ে কী করব?

Running Result-এর সঙ্গে Add

তাই:

Accumulator

Final Pipeline:

Traversal

↓

Accumulation

↓

Output Sum

Lesson 5 — Sum Problem Code

```
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum += arr[i];
    }

    printf("%d\n", sum);

    return 0;
}
```

এখানে দুইটা Traversal আছে।

প্রথম Traversal:

Input নেওয়া

দ্বিতীয় Traversal:

Sum Calculate করা

Flow:

Input
↓
Store in Array
↓
Traversal
↓
Accumulation
↓
Output

Lesson 6 — সব সময় Array লাগবে?

না।

এই Concept Chapter 1-এ দেখেছি। এবার Practical Example দেখি।

Problem:

Nটি Number Input নিয়ে তাদের Sum বের করো।

আমাদের কি পরে Number-গুলো আবার দরকার?

No

তাহলে:

Read
↓
Process
↓
Forget

Code:

```
#include <stdio.h>

int main()
{
    int n;
    int x;
    int sum = 0;

    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &x);

        sum += x;
    }

    printf("%d\n", sum);

    return 0;
}
```

Flow:

```
Read Number
  ↓
Add to Sum
  ↓
Read Next Number
  ↓
Add to Sum
  ↓
...
  ↓
Print Final Sum
```

এখানে Array ব্যবহার না করেও Problem Solve হয়েছে।

Lesson 7 — Initialization কেন গুরুত্বপূর্ণ?

দেখো:

```
int sum = 0;
```

এখানে:

```
sum = 0
```

হলো Initialization।

কেন 0 ?

কারণ আমরা Addition করছি।

ধরো শুরুতে:

```
sum = 0
```

তারপর 5 যোগ করলে:

```
0 + 5 = 5
```

Correct।

কিন্তু যদি ভুল করে লিখি:

```
int sum = 10;
```

তাহলে:

Input:

2 3 4

Calculation:

sum = 10

10 + 2 = 12

12 + 3 = 15

15 + 4 = 19

কিন্তু Correct Answer:

9

অর্থাৎ Initial Value ভুল হলে Final Answer-ও ভুল হবে।

Lesson 8 — Counter Pattern কী?

এবার Problem:

Array-তে কতগুলো Even Number আছে?

Array:

3 8 5 12 7 10

আমাদের Sum দরকার না।

আমাদের জানতে হবে:

কতগুলো Number Even?

তাই:

Need Count

প্রথমে:

```
count = 0
```

Traversal:

```
3
```

```
↓
```

```
Even? No
```

```
↓
```

```
count = 0
```

```
8
```

```
↓
```

```
Even? Yes
```

```
↓
```

```
count = 1
```

```
5
```

```
↓
```

```
Even? No
```

```
↓
```

```
count = 1
```

```
12
```

```
↓
```

```
Even? Yes
```

```
↓
```

```
count = 2
```

```
7
```

```
↓
```

```
Even? No
```

```
↓
```

```
count = 2
```

```
10  
↓  
Even? Yes  
↓  
count = 3
```

Final:

```
count = 3
```

Lesson 9 — Counter কী?

সহজ ভাষায়:

যে Variable কোনো ঘটনা কতবার ঘটেছে তা গণনা, তাকে Counter বলা হয়।

Example:

```
int count = 0;
```

Condition True হলে:

```
count++;
```

অর্থাৎ:

```
Previous Count  
+  
1  
↓  
New Count
```

Lesson 10 — Counting Pattern-এর Complete Thinking

Problem:

Count how many positive numbers exist.

Brain Translation:

Step 1 — সব Number দেখতে হবে?

Yes

তাই:

Traversal

Step 2 — কোন Number দরকার?

Positive Number

তাই:

Condition Checking

Condition:

```
arr[i] > 0
```

Step 3 — Positive পেলে কী করব?

Count বাড়াব

তাই:

Counter

Final Pipeline:

Traversal



Condition Checking



Counter



Output Count

Lesson 11 — Counting Problem Code

```
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int count = 0;

    for (int i = 0; i < n; i++)
    {
        if (arr[i] > 0)
        {
            count++;
        }
    }

    printf("%d\n", count);

    return 0;
}
```

Lesson 12 — Accumulator vs Counter

এখানে অনেক Beginner Confused হয়।

দুটোতেই Variable Update হয়।

কিন্তু উদ্দেশ্য আলাদা।

Pattern	কী জমায়?	Example
Accumulator	Value	Sum
Counter	Occurrence	কতগুলো Match

ধরো Array:

2 4 7 8

Even Number-এর **Sum** চাইলে:

2 + 4 + 8 = 14

Variable:

sum = 14

এটা:

Accumulator

Even Number-এর **Count** চাইলে:

2
4
8

মোট:

3

Variable:

count = 3

এটা:

Counter

Lesson 13 — একই Problem-এ Accumulator + Counter

এবার গুরুত্বপূর্ণ অংশ।

Problem:

Positive Number-গুলোর Average বের করো।

Input:

5
10 -5 20 -3 30

Positive Numbers:

10 20 30

Sum:

60

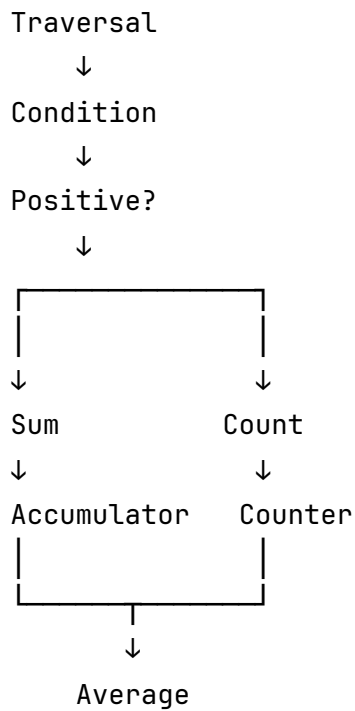
Count:

3

Average:

$60 / 3 = 20$

এখানে কী কী লাগল?



Code Logic:

```
int sum = 0;  
int count = 0;  
  
for (int i = 0; i < n; i++)  
{  
    if (arr[i] > 0)  
    {  
        sum += arr[i];  
        count++;  
    }  
}
```

তারপর:

```
double average = (double)sum / count;
```

এখানে (double) কেন লাগছে, সেটা Data Type এবং Division-এর lesson-এ আরও detail-এ আসবে। আপাতত মনে রাখো integer division যেন decimal অংশ কেটে না ফেলে।

আর একটি Edge Case:

Positive Number একটাও না থাকলে

count = 0

তখন:

sum / count

করা যাবে না।

কারণ Division by Zero হবে।

তাই আগে Check করতে হবে:

```
if (count > 0)
{
    double average = (double)sum / count;
}
```

Lesson 14 — Conditional Accumulation

সব সময় সব Number যোগ করতে হবে না।

Problem:

শুধু Even Number-গুলোর Sum বের করো।

Input:

3 8 5 12 7 10

Even:

8 12 10

Sum:

Thinking:

Traversal
↓
Condition
↓
Even?
↓
Accumulator

Code:

```
int sum = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] % 2 == 0)
    {
        sum += arr[i];
    }
}
```

এটাকে আমরা বলতে পারি:

Conditional Accumulation

অর্থাৎ সব Value না, Condition Match করা Value-গুলো Result-এ জমা হচ্ছে।

Lesson 15 — Conditional Counting

Problem:

কতগুলো Number 100 -এর চেয়ে বড়?

Input:

50 120 30 200 101

Check:

50 > 100? No

120 > 100? Yes → count = 1

30 > 100? No

200 > 100? Yes → count = 2

101 > 100? Yes → count = 3

Pipeline:

Traversal

↓

Condition

↓

Counter

Code:

```
int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] > 100)
    {
        count++;
    }
}
```


Lesson 16 — One Traversal, Multiple Results

একটা Array থেকে একই Traversal-এ একাধিক Result বের করা সম্ভব।

Problem:

Count Even and Odd Numbers.

Input:

1 2 3 4 5 6

Expected:

Even = 3

Odd = 3

Mental Model:

Traversal

↓

Even?

↙

↘

Yes

No

↓

↓

Even++

Odd++

Code:

```
int even = 0;
int odd = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] % 2 == 0)
    {
        even++;
    }
    else
    {
        odd++;
    }
}
```

এখানে দুইটা Counter আছে:

```
even
odd
```

কিন্তু Traversal একবারই।

Lesson 17 — Common Bug: Counter ভুল জায়গায় রাখা

Problem:

Count Even Numbers.

ভুল Code:

```
for (int i = 0; i < n; i++)  
{  
    int count = 0;  
  
    if (arr[i] % 2 == 0)  
    {  
        count++;  
    }  
}
```

Problem কোথায়?

প্রতিটি Iteration-এ:

```
count = 0
```

আবার তৈরি হচ্ছে।

Flow:

```
Iteration 1
```

```
count = 0
```

```
↓
```

```
Maybe count = 1
```

```
↓
```

```
Variable শেষ
```

তারপর:

```
Iteration 2
```

```
count = 0
```

আবার Reset।

Correct:

```
int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] % 2 == 0)
    {
        count++;
    }
}
```

Rule:

```
Initialize Once
  ↓
Loop Many Times
  ↓
Update Result
  ↓
Print After Loop
```

Lesson 18 — Common Bug: Result Loop-এর ভিতরে Print করা

ধরো Sum Problem।

ভুল:

```
int sum = 0;

for (int i = 0; i < n; i++)
{
    sum += arr[i];

    printf("%d\n", sum);
}
```

Input:

1 2 3

Output হবে:

1
3
6

কিন্তু Problem যদি শুধু Final Sum চায়:

6

তাহলে printf() Loop-এর বাইরে হবে:

```
int sum = 0;

for (int i = 0; i < n; i++)
{
    sum += arr[i];
}

printf("%d\n", sum);
```

এখানে গুরুত্বপূর্ণ প্রশ্ন:

Problem কি Running Result চাইছে, নাকি Final Result?

সাধারণ Sum Problem:

Final Result

তাই:

Loop
↓
Finish Processing
↓
Print

Lesson 19 — Common Bug: = এবং +=

এই দুইটা এক না।

ধরো:

```
sum = 10  
arr[i] = 5
```

যদি লিখো:

```
sum = arr[i];
```

তাহলে:

```
sum = 5
```

পুরোনো 10 হারিয়ে গেল।

কিন্তু:

```
sum += arr[i];
```

মানে:

```
sum = 10 + 5
```

Result:

```
15
```

Accumulation-এর জন্য সাধারণত দরকার:

```
sum += value;
```

Lesson 20 — Problem Statement Translation

এখন কিছু common signal চিনে রাখো।

Problem-এ যদি থাকে:

Find the total...

Brain:

Accumulator

যদি থাকে:

Find the sum...

Brain:

Accumulator

যদি থাকে:

How many...

Brain:

Counter

যদি থাকে:

Count the number of...

Brain:

Condition

↓

Counter

যদি থাকে:

Sum of all even numbers...

Brain:

Traversal

↓

Condition

↓

Accumulator

যদি থাকে:

Count all even numbers...

Brain:

Traversal

↓

Condition

↓

Counter

শুধু একটি শব্দ বদলেছে:

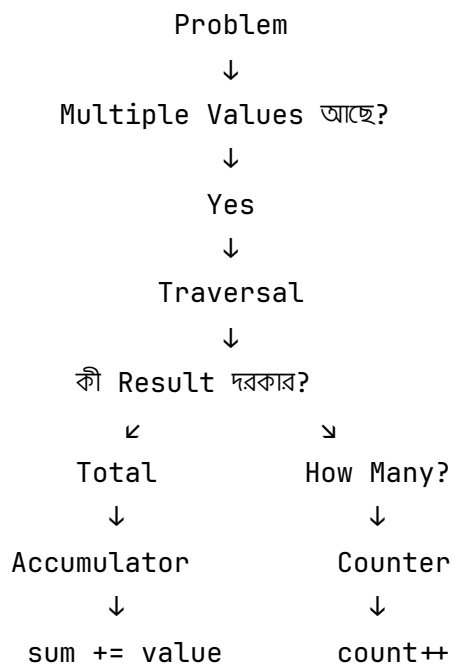
Sum

versus:

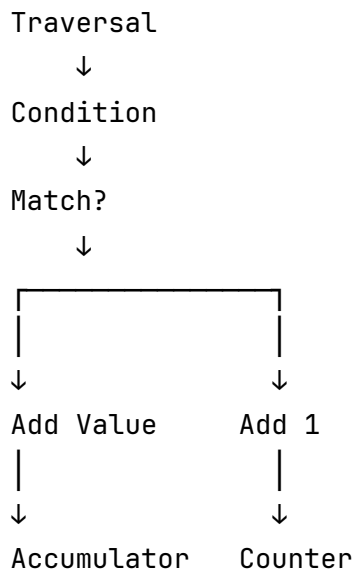
Count

কিন্তু Solution Pattern বদলে গেছে।

Accumulation vs Counting Decision Map



Condition থাকলে:



Worked Example 1 — Sum of Negative Numbers

Problem:

Find the sum of all negative numbers.

Input:

6
5 -2 7 -4 -3 10

Thinking

সব Number দেখতে হবে:

Traversal

শুধু Negative দরকার:

Condition

Negative Value যোগ করতে হবে:

Accumulator

Final Pipeline:

Traversal

↓

Condition Checking

↓

Accumulator

Dry Run:

Value	Negative?	Sum
5	No	0
-2	Yes	-2
7	No	-2
-4	Yes	-6
-3	Yes	-9
10	No	-9

Final:

-9



Worked Example 2 — Count Multiples of 5

Problem:

Count how many numbers are divisible by 5.

Input:

6
10 7 15 8 20 21

Thinking:

Traversal
↓
Divisibility Check
↓
Counter

Condition:

`arr[i] % 5 == 0`

Dry Run:

Value	Divisible by 5?	Count
10	Yes	1
7	No	1
15	Yes	2
8	No	2
20	Yes	3
21	No	3

Final:

3



Chapter 3 Exercise

আজকের pacing ঠিক রাখতে **৬টি focused exercise** থাকবে।

প্রথমে Thinking Pipeline লিখবে, তারপর Code করবে।

Exercise 1 — Sum of All Elements

Problem:

Given N integers, find their total sum.

Input:

```
5
10 20 30 40 50
```

Expected Output:

```
150
```

নিজে লিখবে:

Pattern:

```
???
```

↓

```
???
```

Exercise 2 — Count Positive Numbers

Input:

6
-2 5 0 -7 9 3

Expected Output:

3

নিজে Pipeline লিখবে।

Exercise 3 — Sum of Even Numbers

Input:

6
3 8 5 12 7 10

Expected Output:

30

Hint:

Traversal
↓
???
↓
???

Exercise 4 — Count Numbers Divisible by Both 3 and 5

Input:

6

15 9 10 30 7 45

Expected Output:

3

নিজে Condition তৈরি করবে।

Exercise 5 — Count Even and Odd

Input:

7

1 2 3 4 5 6 8

Expected Output:

Even: 4

Odd: 3

এখানে:

One Traversal

↓

Two Counters

ব্যবহার করবে।

Exercise 6 — Challenge

Problem:

Find the average of all positive numbers.

Input:

6
10 -5 20 -3 30 40

Positive Values:

10 20 30 40

Sum:

100

Count:

4

Average:

25.00

এখানে নিজে Pattern Pipeline তৈরি করবে।



Chapter 3 Assignment

এই Assignment-এ Code লিখবে না। শুধু **Pattern Recognition + Explanation**।

Task 1

Problem:

Find the sum of all numbers greater than 100.

লিখবে:

Pattern:

???

↓

???

↓

???

Why?

...

Task 2

Problem:

Count how many characters in a string are uppercase letters.

লিখবে:

Pattern:

???

↓

???

↓

???

Why?

...

Task 3

Problem:

Given N integers, count positive, negative and zero values separately.

লিখবে:

Pattern:

???

↓

???

↓

???

Variables Needed:

???

Task 4

Problem:

Find the sum and count of all numbers divisible by 7.

এখানে একই Traversal-এর মধ্যে দুই ধরনের Result রাখতে হবে।

নিজে লিখবে:

Pattern:

???

↓

???

↓

???

এবং:

Variables Needed:

???

Task 5 — Concept Check

নিজের ভাষায় উত্তর দেবে:

Accumulator এবং Counter-এর মধ্যে পার্থক্য কী?

Code Example দিয়ে বোঝাবে।



Chapter 3 Quick Revision Sheet

SUM / TOTAL



Accumulator

HOW MANY / COUNT



Counter

Sum of All



Traversal



Accumulator

Count Matching Values



Traversal



Condition



Counter

Sum Matching Values



Traversal



Condition



Accumulator

Average of Matching Values



Traversal



Condition



Accumulator + Counter



sum / count



Chapter 3 Final Mental Model

আজ থেকে Problem দেখলে:

"Sum"

দেখে শুধু Code মনে করবে না।

Brain বলবে:

Need Traversal?



Yes

Need Total?



Accumulator

আর:

"How many?"

দেখলে Brain বলবে:

Need Traversal

↓

Need Condition

↓

Need Counter

Chapter 2 এবং Chapter 3 মিলিয়ে এখন তোমার Pattern Vocabulary দাঁড়ালো:

Traversal

|

— Output

— Condition → Output

— Accumulation

— Condition → Accumulation

— Condition → Counter

— Condition → Accumulator + Counter

এটাই Day 2-এর মূল foundation। **Chapter 4-এ Maximum + Minimum Pattern** আসবে—সেখানে আমরা বিশেষভাবে `max = 0` bug, negative input, initialization, comparison এবং update logic নিয়ে কাজ করব।

Day 2 — Chapter 4



Maximum & Minimum Pattern

Chapter 2-এ আমরা শিখেছি:

Traversal

Chapter 3-এ শিখেছি:

Traversal

↓

Accumulation

এবং:

Traversal

↓

Condition

↓

Counter

এখন আমরা আরেকটি অত্যন্ত গুরুত্বপূর্ণ Pattern শিখব:

Traversal

↓

Comparison

↓

Update

এই Pattern ব্যবহার করে আমরা বের করতে পারি:

Maximum

Minimum

Largest

Smallest

Highest

Lowest

Best

Worst

আজকের Chapter-এর সবচেয়ে গুরুত্বপূর্ণ বিষয় শুধু Maximum বা Minimum-এর Code শেখা নয়।

মূল লক্ষ্য হলো এই চিন্তাটা তৈরি করা:

Current Best Result



New Value-এর সঙ্গে Compare



Better?



Yes



Update



No



Ignore



Chapter Goal

এই Chapter শেষে তুমি বুঝতে পারবে:

- Maximum Pattern কী;
- Minimum Pattern কী;
- Comparison + Update কীভাবে কাজ করে;
- Candidate বলতে কী বোঝায়;
- Current Best কী;
- কেন $\text{max} = 0$ Dangerous;
- কেন $\text{min} = 0$ -ও Dangerous;
- Safe Initialization কী;
- কেন Maximum Traversal অনেক সময় $i = 1$ থেকে শুরু হয়;
- Conditional Maximum কী;
- Maximum Value এবং Maximum Index-এর পার্থক্য;
- Multiple Maximum থাকলে কীভাবে চিন্তা করতে হয়;
- Second Maximum নিয়ে প্রাথমিক ধারণা;
- Common Max/Min Bugs।

Core Mental Model:

Need Best Value
↓
Take a Valid Starting Candidate
↓
Traverse Remaining Values
↓
Compare
↓
Better Candidate Found?
↓
Update Current Best

Lesson 1 — Maximum Problem কী?

ধরো Array:

5 8 2 12 7

Problem:

Largest Number বের করো।

মানুষ হিসেবে আমরা দেখি:

5
↓
8
↓
2
↓
12
↓
7

এবং বুঝি:

Maximum = 12

কিন্তু Computer-কে একটা Process দিতে হবে।

Process:

প্রথমে 5-কে Maximum ধরো

`max = 5`

তারপর:

`8 > 5?`

Yes

`max = 8`

তারপর:

`2 > 8?`

No

`max = 8`

তারপর:

`12 > 8?`

Yes

`max = 12`

তারপর:

`7 > 12?`

No

`max = 12`

Final:

`Maximum = 12`

Lesson 2 — Maximum Pattern

Maximum বের করার মূল Pattern:

```
Take Initial Candidate
    ↓
Traversal
    ↓
Compare Current Element with Maximum
    ↓
Is Current Element Larger?
    ↓
    Yes
    ↓
Update Maximum
```

Code Logic:

```
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

Final Output:

```
printf("%d\n", max);
```

Lesson 3 — Current Best এবং Candidate

এই দুইটা Concept বুঝলে Maximum/Minimum Pattern অনেক সহজ হয়ে যায়।

ধরো:

5 8 2 12 7

প্রথমে:

Current Best = 5

পরের Value:

Candidate = 8

Compare:

$8 > 5$

তাই:

Current Best = 8

পরের Candidate:

2

Compare:

$2 > 8?$

No

তাই:

Current Best এখনও 8

এভাবে:

Current Best
↓
Candidate আসে
↓
Compare
↓
Better হলে Update

এই Mental Model শুধু Maximum-এর জন্য না।

ভবিষ্যতে আরও অনেক Problem-এ কাজে লাগবে।

যেমন:

Highest Score
Lowest Cost
Shortest Distance
Best Profit
Longest Length
Earliest Time
Latest Time

সবক্ষেত্রে Core Idea:

Current Best
↓
Compare Candidate
↓
Update if Better

Lesson 4 — Maximum Dry Run

Array:

4 9 3 15 8

Initialization:

```
max = arr[0]
max = 4
```

Traversal শুরু:

```
i = 1
```

Dry Run:

i	arr[i]	Current max	Comparison	New max
1	9	4	9 > 4 → Yes	9
2	3	9	3 > 9 → No	9
3	15	9	15 > 9 → Yes	15
4	8	15	8 > 15 → No	15

Final:

```
max = 15
```

Lesson 5 — কেন $i = 1$ থেকে শুরু?

দেখো:

```
int max = arr[0];
```

আমরা ইতোমধ্যে প্রথম Element:

```
arr[0]
```

কে Maximum হিসেবে ব্যবহার করেছি।

তাই Traversal:

```
for (int i = 1; i < n; i++)
```

থেকে শুরু করা যায়।

কারণ:

```
arr[0]
```

ইতোমধ্যে Process হয়েছে।

Flow:

```
arr[0]
```

↓

Initial Maximum

```
arr[1]
```

```
arr[2]
```

```
arr[3]
```

```
...
```

↓

Compare with Maximum

তবে যদি $i = 0$ থেকেও শুরু করো:

```
int max = arr[0];
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
    if (arr[i] > max)
```

```
    {
```

```
        max = arr[i];
```

```
    }
```

```
}
```

এটাও Correct।

শুধু প্রথম Element নিজের সঙ্গেই Compare হবে:

```
arr[0] > arr[0]?
```

যা unnecessary, কিন্তু Wrong নয়।

Preferred:

```
for (int i = 1; i < n; i++)
```

Lesson 6 — Complete Maximum Code

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int max = arr[0];
```

```
    for (int i = 1; i < n; i++)
```

```
    {
```

```
        if (arr[i] > max)
```

```
        {
```

```
            max = arr[i];
```

```
        }
```

```
    }
```

```
    printf("%d\n", max);
```

```
    return 0;
```

```
}
```

Thinking Pipeline:

Need Largest Value

↓

Traversal

↓

Comparison

↓

Maximum Update

↓

Output

Lesson 7 — The Dangerous `max = 0` Bug

এটা খুব গুরুত্বপূর্ণ।

Beginner-রা অনেক সময় লেখে:

```
int max = 0;
```

তারপর:

```
for (int i = 0; i < n; i++)  
{  
    if (arr[i] > max)  
    {  
        max = arr[i];  
    }  
}
```

কিছু Input-এর জন্য এটা কাজ করবে।

Input:

```
5  
2 8 3 10 4
```


Output:

10

তখন মনে হবে Code ঠিক।

কিন্তু এবার Input:

5
-5 -2 -8 -1 -9

Actual Maximum:

-1

কিন্তু তোমার Code-এ:

max = 0

Check:

-5 > 0? No

-2 > 0? No

-8 > 0? No

-1 > 0? No

-9 > 0? No

তাই Final:

max = 0

কিন্তু 0 তো Array-তেই নেই!

Correct Answer:

-1

Wrong Answer:

0

Lesson 8 — Safe Initialization

Maximum-এর জন্য Safe Approach:

```
int max = arr[0];
```

কেন?

কারণ:

```
arr[0]
```

নিশ্চিতভাবে Input Data-এর একটি Valid Value।

Array:

-5 -2 -8 -1 -9

Initialization:

```
max = -5
```

তারপর:

-2 > -5?

Yes

```
max = -2
```

তারপর:

`-8 > -2?`

No

তারপর:

`-1 > -2?`

Yes

`max = -1`

Correct!

Rule:

Best Value Problem-এ সম্ভব হলে একটি Valid Input Value দিয়ে Initialize করো।

Lesson 9 — Minimum Pattern

Maximum-এর মতোই Minimum।

Array:

`5 8 2 12 7`

Problem:

Smallest Number বের করো।

প্রথমে:

`min = 5`

তারপর:

8 < 5?

No

2 < 5?

Yes

min = 2

12 < 2?

No

7 < 2?

No

Final:

Minimum = 2

Code:

```
int min = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] < min)
    {
        min = arr[i];
    }
}
```

Pattern:

Traversal

↓

Comparison

↓

Minimum Update

Lesson 10 — `min = 0` কেন Dangerous?

ধরো:

```
int min = 0;
```

Input:

```
5
4 8 2 10 7
```

Actual Minimum:

```
2
```

কিন্তু Check:

```
4 < 0? No
```

```
8 < 0? No
```

```
2 < 0? No
```

```
10 < 0? No
```

```
7 < 0? No
```

Final:

```
min = 0
```

Wrong!

কারণ 0 Array-এর অংশই না।

Safe:

```
int min = arr[0];
```

Lesson 11 — Maximum এবং Minimum একসঙ্গে

Problem:

Find both Maximum and Minimum.

Array:

5 8 2 12 7

আমরা একই Traversal-এ দুটো Result বের করতে পারি।

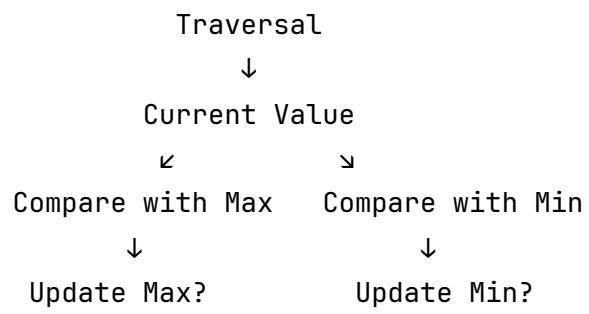
Initialization:

```
int max = arr[0];  
int min = arr[0];
```

Traversal:

```
for (int i = 1; i < n; i++)  
{  
    if (arr[i] > max)  
    {  
        max = arr[i];  
    }  
  
    if (arr[i] < min)  
    {  
        min = arr[i];  
    }  
}
```

Mental Model:



Complete Code:

```
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int max = arr[0];
    int min = arr[0];

    for (int i = 1; i < n; i++)
    {
        if (arr[i] > max)
        {
            max = arr[i];
        }

        if (arr[i] < min)
        {
            min = arr[i];
        }
    }

    printf("Maximum: %d\n", max);
    printf("Minimum: %d\n", min);

    return 0;
}
```


Lesson 12 — Maximum Value vs Maximum Index

ধরো Array:

Index: 0 1 2 3 4

Value: 5 8 2 12 7

Maximum Value:

12

Maximum Value-এর Index:

3

দুটো আলাদা Result।

শুধু Maximum Value দরকার

```
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

Maximum-এর Index-ও দরকার

```
int max = arr[0];
int max_index = 0;

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
        max_index = i;
    }
}
```

যখন Maximum Update হবে:

Maximum Value Update
+
Maximum Index Update

দুটো একসঙ্গে হবে।

Lesson 13 — Index Track করার Dry Run

Array:

4 9 3 15 8

Initial:

max = 4
max_index = 0

Dry Run:

i	Value	Better?	max	max_index
1	9	Yes	9	1
2	3	No	9	1
3	15	Yes	15	3
4	8	No	15	3

Final:

Maximum Value = 15

Maximum Index = 3

Lesson 14 — Duplicate Maximum

Array:

5 12 7 12 3

Maximum:

12

কিন্তু 12 দুইবার আছে।

Indices:

1

3

এখন Problem কী চাইছে সেটা গুরুত্বপূর্ণ।

First Maximum Position

যদি Code হয়:

```
if (arr[i] > max)
```

তাহলে Equal Value পেলে Update হবে না।

Result:

```
First Maximum Index
```

অর্থাৎ:

```
1
```

Last Maximum Position

যদি Code হয়:

```
if (arr[i] ≥ max)
```

তাহলে Equal Maximum পেলেও Update হবে।

Result:

```
Last Maximum Index
```

অর্থাৎ:

```
3
```

এখানে ছোট একটা Operator:

```
>
```

versus:

$\geq$

Problem-এর Result বদলে দিতে পারে।

তাই Statement ভালোভাবে পড়বে।

Lesson 15 — Conditional Maximum

সব সময় পুরো Array-এর Maximum চাইবে না।

Problem:

Find the largest even number.

Input:

5
3 8 5 12 7

Even Values:

8 12

Maximum Even:

12

Thinking:

Traversal
↓
Condition
↓
Even?
↓
Comparison
↓
Maximum Update

এখানে একটি সমস্যা আছে।

আমরা কি লিখব:

```
int max_even = arr[0];
```

না।

কারণ:

```
arr[0]
```

Even নাও হতে পারে।

Input:

```
3 8 10
```

এখানে:

```
arr[0] = 3
```

কিন্তু 3 Valid Candidate না, কারণ আমরা শুধু Even Number-এর মধ্যে Maximum খুঁজছি।

Lesson 16 — Valid Candidate

Initialization

Conditional Maximum-এ আমাদের প্রথম **Valid Candidate** দরকার।

একটি Beginner-friendly Approach:

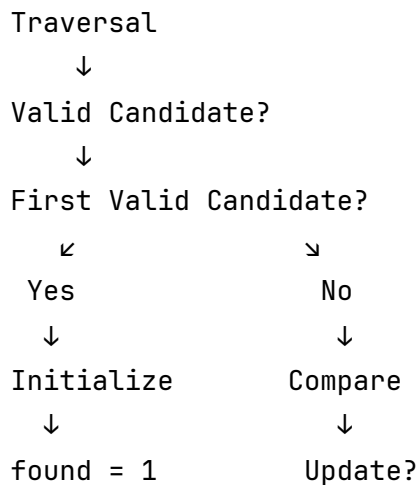
```
int found = 0;
int max_even;

for (int i = 0; i < n; i++)
{
    if (arr[i] % 2 == 0)
    {
        if (found == 0)
        {
            max_even = arr[i];
            found = 1;
        }
        else if (arr[i] > max_even)
        {
            max_even = arr[i];
        }
    }
}
```

শেষে:

```
if (found == 1)
{
    printf("%d\n", max_even);
}
else
{
    printf("No even number\n");
}
```

Mental Model:



এই Pattern ভবিষ্যতে খুব কাজে লাগবে।

Lesson 17 — Maximum Difference: Max - Min

Problem:

Find the difference between the largest and smallest number.

Input:

5
8 2 15 4 10

Maximum:

15

Minimum:

2

Difference:

$$15 - 2 = 13$$

Thinking:

Traversal

↓

Track Maximum + Minimum

↓

max - min

Code Logic:

```
int max = arr[0];
int min = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }

    if (arr[i] < min)
    {
        min = arr[i];
    }
}

int difference = max - min;
```

Lesson 18 — Maximum Input না Maximum Output?

Problem Statement carefully পড়বে।

ধরো:

Find the student with the highest score.

Data:

Student ID: 101 102 103 104

Score: 70 95 80 90

শুধু Maximum Score বের করলে:

95

কিন্তু Problem যদি Student ID চায়:

102

তাহলে শুধু Maximum Value Track করলেই হবে না।

Maximum Update-এর সময় associated information-ও Update করতে হবে।

Mental Model:

Better Score Found

↓

Update Maximum Score

+

Update Student ID

এটাকে এখন Concept হিসেবে মনে রাখো।

পরে Struct এবং Pair-type Problem-এ আরও দেখবে।

Lesson 19 — Second Maximum-এর Basic Idea

এই Chapter-এ আমরা Second Maximum পুরোপুরি Deep Dive করব না, কারণ Day 2-এর scope concise রাখতে হবে।

কিন্তু Concept জানা দরকার।

Array:

5 12 7 9 3

Largest:

12

Second Largest:

9

এখানে শুধু:

max

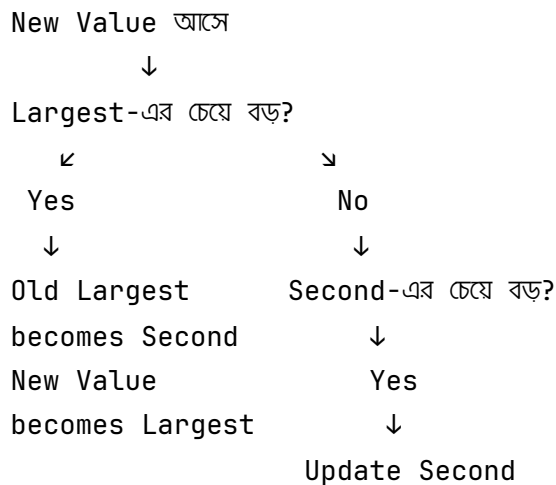
Track করলে হবে না।

আমাদের দুইটি State দরকার:

largest

second_largest

Concept:



এটা Day 2-এর Final Assignment-এর Challenge হিসেবে থাকবে, কিন্তু এখনই Code মুখস্থ করার দরকার নেই।

Lesson 20 — Common Bug: Comparison উল্টো করা

Maximum Problem:

Correct:

```

if (arr[i] > max)
{
    max = arr[i];
}
  
```

ভুল:

```

if (arr[i] < max)
{
    max = arr[i];
}
  
```

এই Code Maximum না, বরং Minimum-এর দিকে যাবে।

মনে রাখো:

Maximum



Current Value > Current Maximum?

Minimum



Current Value < Current Minimum?

Lesson 21 — Common Bug: Wrong Update

ভুল:

```
if (arr[i] > max)
{
    arr[i] = max;
}
```

এখানে তুমি Maximum Update করছো না।

বরং Array Element পরিবর্তন করে দিচ্ছে।

Correct:

```
if (arr[i] > max)
{
    max = arr[i];
}
```

Thinking:

Candidate Better



Current Best = Candidate

অর্থাৎ:

```
max = arr[i]
```

Lesson 22 — Common Bug: Input Constraint না দেখা

ধরো Problem-এর Constraint:

$$-10^9 \leq A[i] \leq 10^9$$

এখন যদি Sum করতে হয়:

```
N = 100000
```

তাহলে Total Sum int range ছাড়িয়ে যেতে পারে।

Maximum/Minimum-এর একটি Element হয়তো int -এ fit করবে, কিন্তু Sum নাও করতে পারে।

তাই Contest-এ Constraint দেখে Data Type নির্বাচন করতে হবে।

উদাহরণ:

```
long long sum = 0;
```

এই Chapter Maximum/Minimum নিয়ে হলেও Rule মনে রাখবে:

Problem Statement

↓

Constraints

↓

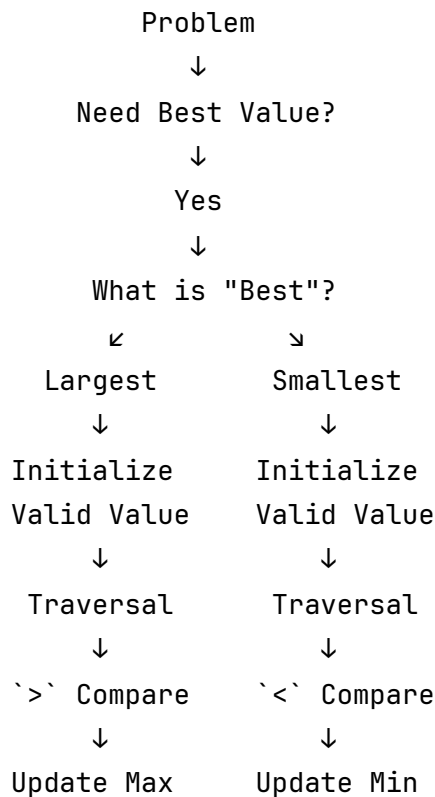
Choose Data Type

↓

Write Logic



Maximum vs Minimum Decision Map



Worked Example 1 — Maximum from Negative Numbers

Problem:

Find the maximum number.

Input:

5
-10 -3 -20 -1 -7

Initialization:

max = -10

Dry Run:

Value	Comparison	Maximum
-3	-3 > -10 → Yes	-3
-20	-20 > -3 → No	-3
-1	-1 > -3 → Yes	-1
-7	-7 > -1 → No	-1

Final:

-1

এখানেই বোঝা যায়:

```
int max = 0;
```

কেন ভুল হতে পারে।

Worked Example 2 — Minimum and Its Index

Problem:

Find the minimum value and its index.

Input:

```
5
8 3 10 1 7
```

Thinking:

Traversal
↓
Comparison
↓
Smaller?
↓
Update Minimum
+
Update Index

Code:

```
int min = arr[0];  
int min_index = 0;  
  
for (int i = 1; i < n; i++)  
{  
    if (arr[i] < min)  
    {  
        min = arr[i];  
        min_index = i;  
    }  
}
```

Result:

Minimum = 1
Index = 3

Chapter 4 Exercise

আজকের pacing অনুযায়ী **৬টি focused exercise** থাকবে।

প্রতিটি Problem-এ:

1. Pattern লিখবে
2. Own Example বানাবে
3. Dry Run করবে
4. তারপর Code করবে

Exercise 1 — Find Maximum

Problem:

Given N integers, find the largest value.

Input:

```
5
7 2 15 4 10
```

Expected Output:

```
15
```

Pattern:

```
???
```

↓

```
???
```

↓

```
???
```

Exercise 2 — Find Minimum

Input:

```
6
8 3 12 1 9 5
```

Expected Output:

1

নিজে Pattern লিখবে।

Exercise 3 — Find Maximum and Minimum Together

Input:

7
10 3 25 7 1 18 9

Expected Output:

Maximum: 25
Minimum: 1

Requirement:

One Traversal
↓
Track Two Results

Exercise 4 — Maximum Index

Input:

6
4 10 7 25 3 8

Expected Output:

Maximum: 25

Index: 3

Requirement:

Maximum Update

+

Index Update

Exercise 5 — Difference Between Maximum and Minimum

Input:

5

8 2 15 4 10

Expected Output:

13

নিজে Pipeline তৈরি করবে।

Exercise 6 — Challenge: Largest Even Number

Input:

7

3 8 5 12 7 20 9

Expected Output:

Pattern Hint:

Traversal

↓

Condition

↓

Valid Candidate Initialization

↓

Comparison

↓

Update

আর এই Input-টাও Test করবে:

5

3 5 7 9 11

কারণ এখানে কোনো Even Number নেই।



Chapter 4 Assignment

Assignment-এ প্রথমে Code লিখবে না।

Pattern Recognition এবং Explanation লিখবে।

Task 1

Problem:

Find the smallest negative number.

উদাহরণ:

5 -2 7 -10 -3

Answer:

-10

লিখবে:

Pattern:

???

↓

???

↓

???

↓

???

Why?

...

সতর্কতা: “smallest negative” মানে zero-এর কাছাকাছি negative না; numeric value হিসেবে সবচেয়ে ছোটটি।

Task 2

Problem:

Find the largest positive number.

লিখবে:

Pattern:

```
???  
↓  
???  
↓  
???  
↓  
???
```

Initialization Problem:

...

Solution Idea:

...

Task 3

Problem:

Find Maximum Value and its first occurrence index.

Array:

```
5 12 7 12 3
```

Expected:

```
Maximum = 12  
Index = 1
```

প্রস্ন:

Comparison-এ `>` ব্যবহার করবে নাকি `≥`?

কারণ:

...

Task 4

Problem:

Find Maximum Value and its last occurrence index.

একই Array:

5 12 7 12 3

Expected:

Maximum = 12

Index = 3

প্রশ্ন:

Comparison কী হবে?

কারণ:

...

Task 5 — Bug Hunt

এই Code-এর Problem খুঁজে বের করো:


```
int max = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

লিখবে:

Bug:

...

Failing Test Case:

...

Expected Output:

...

Actual Output:

...

Fix:

...

Task 6 — Challenge Thinking

Problem:

Find the second largest distinct number.

Input:

7

5 12 7 12 9 3 8

Expected:

9

এখন Code না পারলেও সমস্যা নেই।

শুধু লিখবে:

What states do I need?

1. ???
2. ???

What happens when a new largest value appears?

...

What happens when a value is smaller than largest
but greater than second largest?

...



Chapter 4 Quick Revision Sheet

Maximum

↓

Traversal

↓

Comparison: `>`

↓

Update Max

Minimum



Traversal



Comparison: ` $<$ `



Update Min

Maximum + Index



Traversal



Comparison



Update Value + Index

Conditional Maximum



Traversal



Condition



Find Valid Candidate



Comparison



Update

First Maximum Position



Usually ` $>$ `

Last Maximum Position



Usually ` $\geq$ `



Chapter 4 Final Mental Model

আজ থেকে:

Largest
Highest
Maximum
Best Score

দেখলে Brain:

Need Best Candidate
↓
Valid Initialization
↓
Traversal
↓
Comparison
↓
Update Maximum

আর:

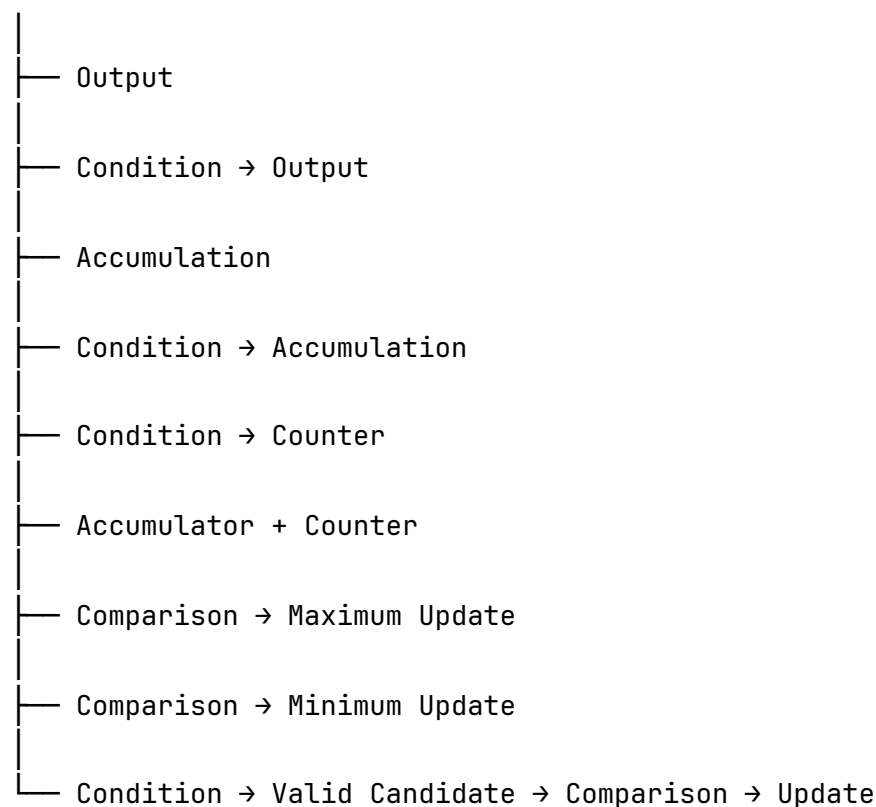
Smallest
Lowest
Minimum
Cheapest

দেখলে Brain:

Need Best Candidate
↓
Valid Initialization
↓
Traversal
↓
Comparison
↓
Update Minimum

এখন পর্যন্ত Day 2-তে তোমার Pattern Map:

Traversal



Day 2 — Chapter 4 এখানেই শেষ। পরের Chapter হবে **Chapter 5 — Linear Search Pattern।** সেখানে target , found flag , break , first occurrence, last occurrence, count occurrences এবং search-এর common bugs একসঙ্গে শেখা হবে।

Day 2 — Chapter 5



Linear Search Pattern

Chapter 2-এ আমরা শিখেছি:

Traversal

Chapter 3-এ:

Traversal



Accumulation / Counting

Chapter 4-এ:

Traversal



Comparison



Update Best Result

এখন আমরা শিখব আরেকটি অত্যন্ত গুরুত্বপূর্ণ Pattern:

Traversal



Target Comparison



Match Found?

এই Pattern-এর নাম:

Linear Search



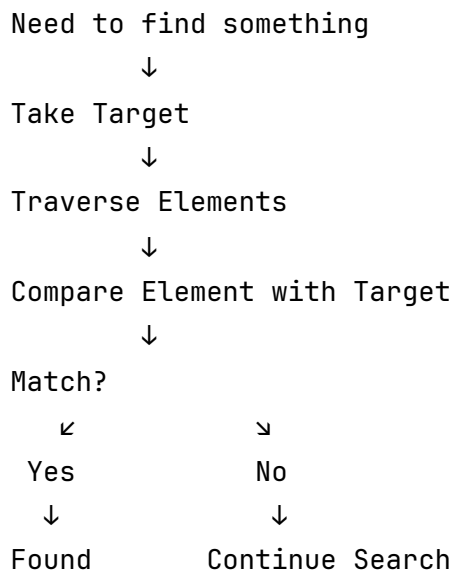
Chapter Goal

এই Chapter শেষে তুমি বুঝতে পারবে:

- Search Problem কী;
- Linear Search কী;
- Target কী;
- Match কী;
- found Flag কী;
- Flag কেন ব্যবহার করা হয়;
- break কেন এবং কখন ব্যবহার করতে হয়;

- Early Termination কী;
- First Occurrence কী;
- Last Occurrence কী;
- Count Occurrence কী;
- Value Search এবং Index Search-এর পার্থক্য;
- break দিলে Result কীভাবে বদলে যায়;
- Search এবং Counting-এর সম্পর্ক;
- Linear Search-এর Time Complexity কেন $O(n)$;
- Common Search Bugs কী।

আজকের Core Mental Model:



Lesson 1 — Search Problem কী?

ধরো Array:

5 8 2 12 7

Problem:

Array-তে 12 আছে কি না বের করো।

এখানে:

Target = 12

আমরা Check করব:

5 = 12?

No

8 = 12?

No

2 = 12?

No

12 = 12?

Yes

আমরা Target পেয়ে গেছি।

Result:

Found

এই Process হলো:

Traversal

↓

Target Comparison

↓

Match Detection

Lesson 2 — Linear Search কী?

সহজ ভাষায়:

একটি Collection-এর Element-গুলো একে একে Check করে Target খোঁজাকে Linear Search বলে।

ধরো:

Array:

10 20 30 40 50

Target:

40

Search:

10

↓

Match? No

20

↓

Match? No

30

↓

Match? No

40

↓

Match? Yes

Target Found!

এখানে Search একদিক থেকে একটার পর একটা Element Check করছে।

তাই:

Linear

অর্থাৎ:

One by One

Lesson 3 — Target কী?

Search Problem-এ আমরা যেটা খুঁজছি সেটাই:

Target

Example:

Find whether 7 exists in the array.

এখানে:

Target = 7

Problem:

Find the index of 100 .

এখানে:

Target = 100

Problem:

Count how many times 5 occurs.

এখানেও:

Target = 5

অর্থাৎ Target একই ধরনের হতে পারে, কিন্তু Problem-এর Goal আলাদা হতে পারে।

Target Search

- Exists?
- First Position?
- Last Position?
- How Many Times?

এই পার্থক্যটা আজ পরিষ্কার করব।

Lesson 4 — Basic Linear Search

Problem:

Check whether Target exists.

Input:

5
10 20 30 40 50

Target = 30

Basic Logic:

```
for (int i = 0; i < n; i++)  
{  
    if (arr[i] == target)  
    {  
        printf("Found\n");  
    }  
}
```

এখানে একটা সমস্যা আছে।

ধরো Target:

100

কোনো Match পাওয়া গেল না।

তাহলে Code কিছুই Print করবে না।

আমাদের দরকার:

Found

অথবা:

Not Found

এই Problem Solve করতে আমরা ব্যবহার করব:

Flag Variable

Lesson 5 — Flag Variable কী?

Flag হলো একটি Variable যেটা কোনো Condition বা Event-এর State মনে রাখে।

সহজভাবে:

Something happened?

তার উত্তর রাখে:

Yes / No

C-তে Beginner-friendlyভাবে:

0 = No

1 = Yes

Search শুরু করার আগে:

```
int found = 0;
```

মানে:

এখনও Target পাওয়া যায়নি

Target পেলো:

```
found = 1;
```

মানে:

Target পাওয়া গেছে

Flow:

```
found = 0
```

↓

```
Search শুরু
```

↓

```
Target Found?
```

↓

```
found = 1
```

Lesson 6 — Basic Search with Flag

```
#include <stdio.h>

int main()
{
    int n;
    int target;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &target);

    int found = 0;

    for (int i = 0; i < n; i++)
    {
        if (arr[i] == target)
        {
            found = 1;
            break;
        }
    }

    if (found == 1)
    {
        printf("Found\n");
    }
    else
    {
        printf("Not Found\n");
    }
}
```

```
    return 0;  
}
```

Mental Model:

```
Target Input  
  ↓  
found = 0  
  ↓  
Traversal  
  ↓  
arr[i] == target?  
  ↓  
Yes  
  ↓  
found = 1  
  ↓  
break  
  ↓  
Final Decision
```

Lesson 7 — Flag কেন দরকার?

ধরো:

Array:

3 8 5 12 7

Target:

12

Dry Run:

i	arr[i]	Match?	found
0	3	No	0
1	8	No	0
2	5	No	0
3	12	Yes	1

শেষে:

```
found = 1
```

তাই:

```
Found
```

এবার Target:

```
100
```

পুরো Traversal-এর পরেও:

```
found = 0
```

তাই:

```
Not Found
```

Flag মূলত Search-এর Result **মনে রাখছে**।

Lesson 8 — break কী করছে?

এই Code দেখো:


```
if (arr[i] == target)
{
    found = 1;
    break;
}
```

break মানে:

বর্তমান Loop সঙ্গে সঙ্গে বন্ধ করে দাও।

ধরো:

Array:

2 5 7 9 10

Target:

7

Search:

2 → No

5 → No

7 → Yes

এখন Problem শুধু জানতে চায়:

7 আছে কি?

Answer ইতোমধ্যে পাওয়া গেছে।

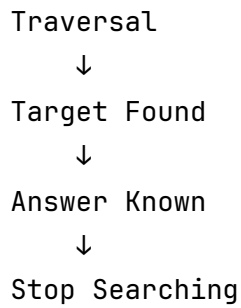
তাই:

9

10

Check করার আর দরকার নেই।

Flow:



এটাই:

Early Termination

Lesson 9 — সব Search-এ break দেওয়া যাবে?

না।

এটা অত্যন্ত গুরুত্বপূর্ণ।

Problem:

| Target আছে কি না?

Target পেলে:

Answer Known

তাই:

`break;`

দেওয়া যায়।

কিন্তু Problem:

| Target কতবার আছে?

Array:

5 2 5 7 5 9

Target:

5

Occurrences:

Index 0

Index 2

Index 4

Total:

3

যদি প্রথম 5 পেয়েই:

`break;`

দাও, তাহলে বাকি 5 -গুলো আর Check হবে না।

তাই:

Exists?

↓

Can Stop Early

Count All?

↓

Must Continue

Lesson 10 — Search Problem-এর বিভিন্ন ধরন

একই Array:

5 12 7 12 3

Target:

12

Problem হতে পারে:

Type 1 — Exists?

12 আছে?

Answer:

Yes

Type 2 — First Occurrence

প্রথম 12 কোথায়?

Answer:

Index 1

Type 3 — Last Occurrence

শেষ 12 কোথায়?

Answer:

Index 3

Type 4 — Count Occurrences

12 কতবার আছে?

Answer:

2

একই Target।

একই Array।

কিন্তু Pattern-এর শেষ অংশ বদলে যাচ্ছে।

Lesson 11 — Existence Search

Problem:

Check whether Target exists.

Pattern:

Traversal



Target Comparison



Match?



Set Flag



Early Termination

Code:

```
int found = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        found = 1;
        break;
    }
}
```

শেষে:

```
if (found)
{
    printf("Found\n");
}
else
{
    printf("Not Found\n");
}
```

এখানে:

```
if (found)
```

এবং:

```
if (found == 1)
```

এই ক্ষেত্রে একই অর্থে ব্যবহার করা যায়।

কারণ:

```
0      → False  
Non-zero → True
```

তবে Bootcamp-এর এই পর্যায়ে clarity-এর জন্য:

```
if (found == 1)
```

লিখলেও সমস্যা নেই।

Lesson 12 — First Occurrence

Array:

```
5 12 7 12 3
```

Target:

```
12
```

First Occurrence:

```
Index 1
```

আমাদের প্রথম Match পাওয়ার পর Search বন্ধ করতে হবে।

Initialization:

```
int index = -1;
```

কেন -1 ?

কারণ Valid Array Index সাধারণত:

0 থেকে $n - 1$

তাই:

-1

মানে:

Not Found

Code:

```
int index = -1;

for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        index = i;
        break;
    }
}
```

Flow:

```
index = -1
↓
Traversal
↓
Target Match?
↓
Yes
↓
Save Index
↓
break
```


Lesson 13 — কেন First Occurrence-এ break ?

Array:

5 12 7 12 3

Search:

Index 0 → 5

Index 1 → 12 → Match

আমরা First Occurrence চাই।

তাই Answer:

1

এরপর Index 3-এর 12 আমাদের দরকার নেই।

তাই:

`break;`

দেওয়া হবে।

Mental Model:

```
First Match
  ↓
Save Index
  ↓
Stop
```

Lesson 14 — Last Occurrence

একই Array:

5 12 7 12 3

Target:

12

Last Occurrence:

Index 3

এবার প্রথম Match পেয়ে থামলে চলবে না।

Code:

```
int index = -1;

for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        index = i;
    }
}
```

খেয়াল করো:

No break

Dry Run:

i	Value	Match?	index
0	5	No	-1
1	12	Yes	1

i	Value	Match?	index
2	7	No	1
3	12	Yes	3
4	3	No	3

Final:

```
index = 3
```

কারণ প্রতিবার Match হলে পুরোনো Index নতুন Index দিয়ে Replace হয়েছে।

Lesson 15 — First vs Last Occurrence

এই পার্থক্য মনে রাখো:

First Occurrence

```
Match
↓
Save Index
↓
Break
```

Code:

```
if (arr[i] == target)
{
    index = i;
    break;
}
```

Last Occurrence

Match

↓

Save Index

↓

Continue Search

↓

Future Match হলে Update

Code:

```
if (arr[i] == target)
{
    index = i;
}
```

মূল পার্থক্য:

First Occurrence

↓

break

Last Occurrence

↓

no break

Lesson 16 — Reverse Search দিয়ে Last Occurrence

Last Occurrence বের করার আরেকটি উপায় আছে।

Array:

5 12 7 12 3

Target:

12

Right থেকে Search:

Index 4 → 3 → No

Index 3 → 12 → Match

এটাই Last Occurrence।

Code:

```
int index = -1;

for (int i = n - 1; i ≥ 0; i--)
{
    if (arr[i] == target)
    {
        index = i;
        break;
    }
}
```

Mental Model:

Reverse Traversal

↓

Target Comparison

↓

First Match from Right

↓

Actually Last Occurrence

এটা Chapter 2-এর Reverse Traversal এবং Chapter 5-এর Search Pattern-এর Combination।

Lesson 17 — Count Occurrences

Problem:

| Target কতবার আছে?

Array:

5 2 5 7 5 9

Target:

5

Thinking:

Need all Elements

↓

Traversal

Need Target Match

↓

Condition

Need How Many

↓

Counter

Final Pattern:

Traversal

↓

Target Comparison

↓

Counter

Code:

```
int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        count++;
    }
}
```

Final:

```
count = 3
```

এখানে তুমি দেখতে পাচ্ছে:

নতুন Problem অনেক সময় সম্পূর্ণ নতুন Pattern নয়; পুরোনো Pattern-এর Combination।

এখানে:

```
Linear Search Thinking
+
Counting Pattern
```

একসঙ্গে কাজ করছে।

Lesson 18 — Search এবং Count-এর পার্থক্য

Problem 1:

7 আছে কি?

আমাদের দরকার:

```
Boolean-like State
```

তাই:

Flag

Problem 2:

7 কতবার আছে?

আমাদের দরকার:

Number of Matches

তাই:

Counter

Comparison:

Problem Goal	Variable
Exists?	found
First Position?	index
Last Position?	index
How Many?	count

Problem Statement-এর Goal বুঝে State Variable ঠিক করবে।

Lesson 19 — Search করে Value না Index?

ধরো:

Index: 0 1 2 3 4

Value: 10 20 30 40 50

Target:

40

Problem বলতে পারে:

| Is 40 present?

Answer:

Yes

অথবা:

| Find the index of 40 .

Answer:

3

তাই Problem পড়ে বুঝতে হবে:

Need Existence?

নাকি:

Need Position?

Code-এর State সে অনুযায়ী বদলাবে।

Lesson 20 — Common Bug: = vs ==

Search-এর সবচেয়ে Dangerous Beginner Bug-এর একটি।

Correct:

```
if (arr[i] == target)
```

মানে:

Compare

কিন্তু:

```
if (arr[i] = target)
```

এখানে Comparison হচ্ছে না।

এখানে Assignment হচ্ছে।

অর্থাৎ:

arr[i]-এর Value বদলে Target বসিয়ে দেওয়া হচ্ছে

মনে রাখবে:

= Assignment

= Equality Comparison

Search Problem:

Need Comparison

তাই:

```
==
```

Lesson 21 — Common Bug: Loop-এর ভিতরে Not Found Print

এই Code ভুল:

```
for (int i = 0; i < n; i++)  
{  
    if (arr[i] == target)  
    {  
        printf("Found\n");  
    }  
    else  
    {  
        printf("Not Found\n");  
    }  
}
```

ধরো:

Array:

5 8 12

Target:

12

Output হবে:

Not Found
Not Found
Found

কিন্তু আমরা চাই:

Found

কেন ভুল হলো?

কারণ:

একটা Element Target না হওয়া

মানে এই না যে:

পুরো Array-তে Target নেই

Not Found বলা যাবে কখন?

পুরো Search শেষ

↓

কোনো Match নেই

↓

Not Found

Correct Mental Model:

Search Entire Array

↓

Track Match State

↓

After Search

↓

Final Decision

Lesson 22 — Common Bug: Flag Reset করা

ভুল:

```
for (int i = 0; i < n; i++)
{
    int found = 0;

    if (arr[i] == target)
    {
        found = 1;
    }
}
```

Problem:

প্রতিটি Iteration-এ
found আবার 0 হচ্ছে

Correct:

```
int found = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        found = 1;
        break;
    }
}
```

Rule:

Initialize State



Before Loop

Update State



Inside Loop

Use Final State



After Loop

Lesson 23 — Common Bug: First Occurrence-এ break না দেওয়া

Problem:

Find first occurrence.

Array:

5 12 7 12 3

Wrong:

```
int index = -1;

for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        index = i;
    }
}
```

Result:

কিন্তু First Occurrence:

1

কারণ Search continue করেছে এবং পরে Index overwrite হয়েছে।

Correct:

```
if (arr[i] == target)
{
    index = i;
    break;
}
```

Lesson 24 — Common Bug: Count Occurrence-এ break

Problem:

Count Target Occurrences.

Wrong:

```
int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        count++;
        break;
    }
}
```

এখানে সর্বোচ্চ:

```
count = 1
```

হতে পারবে।

কারণ প্রথম Match-এর পর Loop বন্ধ।

Count All-এর জন্য:

```
Full Traversal Required
```

তাই:

```
No break
```

Lesson 25 — Linear Search Time Complexity

এখন Complexity খুব Deep-এ যাচ্ছি না।

শুধু Contest-এর জন্য Basic Understanding।

ধরো Array Size:

```
n
```

Worst Case-এ Target হতে পারে:

```
শেষ Element
```

অথবা:

```
Array-তে নেই
```


তখন আমাদের সব Element Check করতে হবে।

n Elements

↓

At most n Checks

তাই Linear Search-এর Worst-case Time Complexity:

$O(n)$

সহজভাবে:

Array Size বাড়লে

Search Work-ও Linearly বাড়ে

Example:

10 Elements

→ At most 10 checks

1000 Elements

→ At most 1000 checks

এখন শুধু এটুকুই মনে রাখবে।

Lesson 26 — Read and Search Without Array?

ধরো Problem:

Nটি Number Input নাও এবং বলো Target আছে কি না।

যদি পরে Number-গুলো দরকার না হয়, theoretically Input নেওয়ার সময়ই Search করা যায়।

Example:

```
int found = 0;
int x;

for (int i = 0; i < n; i++)
{
    scanf("%d", &x);

    if (x == target)
    {
        found = 1;
    }
}
```

কিন্তু এখানে একটি গুরুত্বপূর্ণ বিষয় আছে।

Input stream-এর সব Value পড়তে হতে পারে, বিশেষ করে program structure অনুযায়ী।

তাই:

Target Found

হলেও Input Loop থেকে হঠাৎ break দেওয়া সব situation-এ ভালো design নয়।

Bootcamp-এর জন্য Rule:

যদি Array Problem হিসেবে Data Store করা থাকে, Search Loop-এ break ব্যবহার করো। Input পড়ার Loop এবং Search-এর Logic আলাদা রাখলে Beginner হিসেবে reasoning পরিস্কার থাকবে।

Lesson 27 — Search Pattern Recognition Signals

Problem Statement-এ এই ধরনের শব্দ দেখলে Search চিন্তা করবে:

Find whether...

Check if X exists...

Locate...

Find the position...

Find the first occurrence...

Find the last occurrence...

How many times X appears...

তখন Brain:

Target আছে

↓

Elements inspect করতে হবে

↓

Traversal

↓

Target Comparison

তারপর Goal অনুযায়ী:

Exists?

↓

Flag

First Position?

↓

Index + Break

Last Position?

↓

Index Update

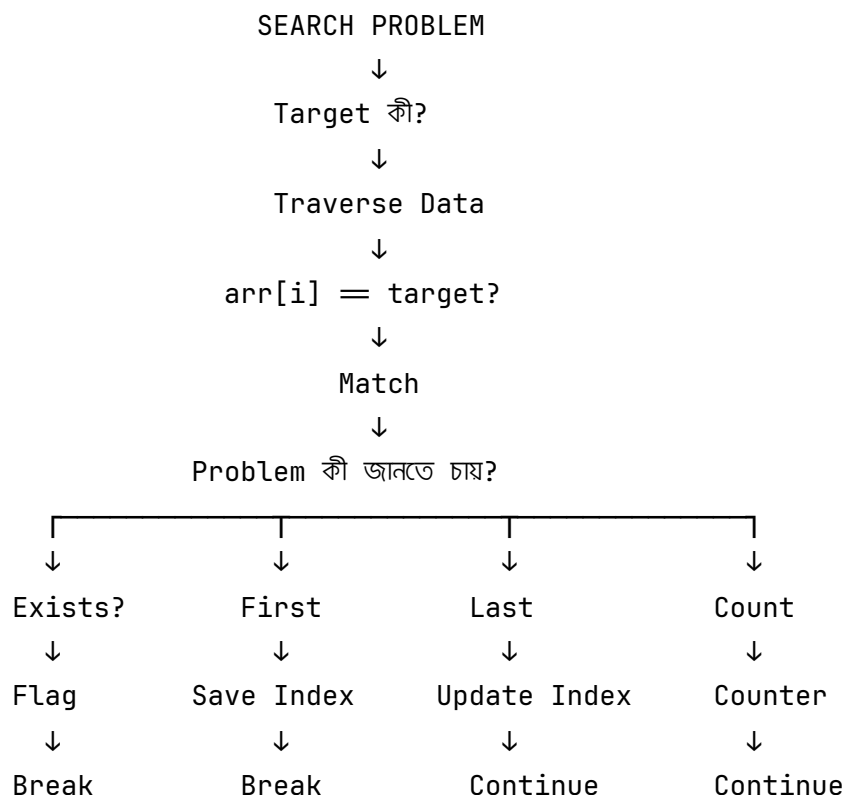
How Many?



Counter



Linear Search Decision Map



Worked Example 1 — Search Target

Problem:

Check whether 7 exists.

Input:

6
3 8 5 7 12 10

Target:

7

Step 1 — Need Search

Target = 7

Step 2 — Need Elements

Traversal

Step 3 — Need Existence Only

Flag

Step 4 — Once Found

Early Termination

Final Pipeline:

Traversal
↓
Target Comparison
↓
Flag Update
↓
Break

Dry Run:

Value	Match?	found
3	No	0
8	No	0
5	No	0
7	Yes	1

তারপর:

break

Final:

Found



Worked Example 2 — Count Occurrences

Problem:

Count how many times 5 appears.

Input:

7
5 2 5 8 5 3 5

Thinking:

Traversal
↓
Target Comparison
↓
Counter

Dry Run:

Value	Match?	Count
5	Yes	1
2	No	1
5	Yes	2
8	No	2
5	Yes	3
3	No	3
5	Yes	4

Final:

4



Worked Example 3 — First Occurrence

Problem:

Find the first index of Target.

Input:

7
3 8 5 8 10 8 2

Target:

8

Thinking:

Traversal

↓

Target Comparison

↓

First Match

↓

Save Index

↓

Break

Dry Run:

Index 0

Value 3

No Match

Index 1

Value 8

Match

তাই:

index = 1

তারপর:

break



Chapter 5 Exercise

আজকের Day 2 pacing বজায় রাখতে **৬টি focused exercise** থাকবে।

প্রতিটি Exercise-এ:

1. Pattern লিখবে
2. Own Test Case বানাবে
3. Dry Run করবে
4. Code লিখবে
5. অন্তত 3টি Test Case চালাবে

Exercise 1 — Exists or Not

Problem:

Check whether Target exists in the array.

Input:

```
5
10 20 30 40 50
30
```

Expected Output:

```
Found
```

আর Test করবে:

```
5
10 20 30 40 50
100
```

Expected:

```
Not Found
```

নিজে Pattern লিখবে।

Exercise 2 — First Occurrence

Input:

```
7
3 8 5 8 10 8 2
8
```

Expected Output:

```
1
```

Requirement:

```
First Match
↓
Save Index
↓
Early Termination
```

Exercise 3 — Last Occurrence

একই Input:

```
7
3 8 5 8 10 8 2
8
```

Expected Output:

```
5
```

এখানে চিন্তা করবে:

```
break দেব?
```

নাকি:

Search Continue করব?

Exercise 4 — Count Occurrences

Input:

```
8
5 2 5 8 5 3 5 10
5
```

Expected Output:

```
4
```

Pattern নিজে লিখবে।

Exercise 5 — Search Negative Target

Input:

```
6
5 -2 8 -10 3 7
-10
```

Expected:

```
Found at index 3
```

এই Exercise-এর উদ্দেশ্য:

Search Logic positive number-এর উপর নির্ভর করে না

Target negative হলেও একই Pattern।

Exercise 6 — Challenge: First and Last Position

Problem:

Target-এর First এবং Last Occurrence বের করো।

Input:

```
8
5 2 7 2 9 2 10 2
2
```

Expected:

```
First: 1
Last: 7
```

Requirement:

```
One Full Traversal
```

Hint:

```
first = -1
last = -1
```

কিন্তু পুরো Logic নিজে তৈরি করবে।



Chapter 5 Assignment

Assignment-এ প্রথমে **Code লিখবে না**। Pattern Recognition, State Selection এবং Reasoning লিখবে।

Task 1 — Existence Search

Problem:

Check whether x exists in an array.

লিখবে:

Pattern:

???

↓

???

↓

???

↓

???

State Variable:

???

Why?

...

Task 2 — First Occurrence

Problem:

Find the first index of x .

লিখবে:

Initial State:

???

Pattern:

???

↓

???

↓

???

↓

???

Why is break needed?

...

Task 3 — Last Occurrence

Problem:

Find the last index of x .

লিখবে:

Initial State:

???

Pattern:

???

↓

???

↓

???

Why should search continue?

...

Task 4 — Count Occurrences

Problem:

Count how many times x appears.

লিখবে:

Pattern:

???

↓

???

↓

???

Variable Needed:

???

Should I use break?

Yes / No

Why?

...

Task 5 — Bug Hunt

এই Code-এর সমস্যা বের করো:

```
for (int i = 0; i < n; i++)  
{  
    if (arr[i] == target)  
    {  
        printf("Found\n");  
    }  
    else  
    {  
        printf("Not Found\n");  
    }  
}
```

লিখবে:

Bug:

...

Why is it wrong?

...

Failing Test Case:

...

Correct Thinking:

...

Task 6 — Bug Hunt

Problem:

Find first occurrence.

Code:

```
int index = -1;

for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
    {
        index = i;
    }
}
```

Answer করবে:

এই Code আসলে কী বের করছে?

...

First Occurrence-এর জন্য কী পরিবর্তন করতে হবে?

...

কেন?

...

Task 7 — Pattern Combination

Problem:

Count how many times the Maximum Value appears.

Example:

7

5 12 7 12 3 12 8

Maximum:

12

Count:

3

এখন Code লিখবে না।

শুধু চিন্তা করো:

Phase 1:

???

Phase 2:

???

তারপর Pattern লিখবে:

???

↓

???

↓

???

↓

???

এটা Chapter 4 এবং Chapter 5-এর Combination।



Important Pattern Combination

এখন পর্যন্ত আমরা Pattern আলাদা করে শিখছি।

কিন্তু বাস্তব Problem-এ এগুলো combine হবে।

উদাহরণ:

Maximum Number কতবার আছে?

Thinking:

Traversal

↓

Comparison

↓

Maximum Find

তারপর:

Traversal



Target Comparison



Counter

অর্থাৎ:

Maximum Pattern

+

Counting/Search Pattern

আবার Problem:

First Even Number-এর Index বের করো।

Thinking:

Traversal



Condition



First Valid Match



Save Index



Break

তাই Pattern Library-কে আলাদা আলাদা Box হিসেবে দেখবে না।

বরং:

Small Patterns



Combine



Solve Bigger Problem



Chapter 5 Quick Revision Sheet

Exists?



Traversal



Target Comparison



Flag



Break

First Occurrence



Traversal



Target Comparison



Save Index



Break

Last Occurrence



Traversal



Target Comparison



Update Index



Continue

Count Occurrences



Traversal



Target Comparison



Counter



No Break

Search Result State

Exists? → found

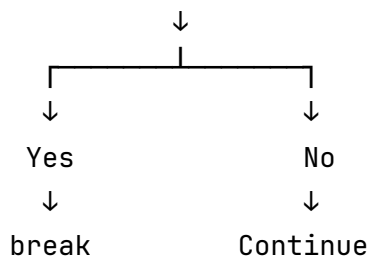
Position? → index

How Many? → count

break Decision Rule

এটা খুব ভালো করে মনে রাখবে:

Match পাওয়ার পর Answer সম্পূর্ণ জানা গেছে?



Examples:

Problem	break ?
Target exists?	Yes
First occurrence	Yes
Last occurrence, forward scan	No

Problem	break ?
Count occurrences	No
Print all matches	No

এটা মুখস্থ করার চেয়ে Rule-টা বোঝো:

Answer complete হলে stop; future elements answer বদলাতে পারলে continue।



Chapter 5 Final Mental Model

আজ থেকে Search Problem দেখলে প্রথমে Code চিন্তা করবে না।

নিজেকে জিজ্ঞেস করবে:

আমি কী খুঁজছি?



Target

তারপর:

কোথায় খুঁজছি?



Collection / Array

তারপর:

সব Element Check করতে হবে?



Traversal

তারপর সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন:

Problem কী Result চাইছে?

তার ভিত্তিতে:

Exists?



Flag

First Position?



Index + Break

Last Position?



Index Update + Continue

How Many?



Counter



Day 2 Progress

এখন পর্যন্ত আমরা শেষ করেছি:

Day 2



Chapter 1 – Array Mental Model



Chapter 2 – Traversal Pattern



Chapter 3 – Accumulation + Counting



Chapter 4 – Maximum + Minimum



Chapter 5 – Linear Search



Chapter 6 – Mixed Pattern Problems



Chapter 7 – Mistake Review + Pattern Library + Final Assignment

Day 2-এর core theory এখন শেষ। **Chapter 6 থেকে নতুন theory অনেক কম থাকবে**—সেখানে আসল কাজ হবে Problem দেখে নিজে ঠিক করা:

Traversal?
Accumulator?
Counter?
Maximum?
Minimum?
Search?
Flag?
Index?
Break?

অর্থাৎ Chapter 6 হবে আজ পর্যন্ত শেখা Pattern-গুলো **mix করে problem-solving practice**, আর Chapter 7 দিয়ে Day 2 close হবে।

Day 2 — Chapter 6



Mixed Pattern Problems

Chapter 1 থেকে Chapter 5 পর্যন্ত আমরা একেকটি Core Pattern আলাদাভাবে শিখেছি।

Chapter 1



Array Mental Model

Chapter 2



Traversal

Chapter 3



Accumulation + Counting

Chapter 4



Maximum + Minimum

Chapter 5



Linear Search

এখন পর্যন্ত Problem দেখলে Pattern অনেকটা পরিষ্কার ছিল।

যেমন:

Find Sum



Accumulation

অথবা:

Find Maximum



Comparison + Update

কিন্তু Contest-এর Problem সাধারণত সব সময় এত সরাসরি হবে না।

একটা Problem-এর মধ্যে থাকতে পারে:

Traversal
+
Condition
+
Counter
+
Maximum

অথবা:

Maximum
+
Search
+
Counter

অথবা:

Condition
+
Accumulator
+
Counter
+
Average

তাই Chapter 6-এর লক্ষ্য নতুন Pattern শেখা না।

লক্ষ্য হলো:

ছোট ছোট Pattern Combine করে বড় Problem Solve করা।

Chapter Goal

এই Chapter শেষে তুমি বুঝতে পারবে:

- Mixed Pattern Problem কী;
- Problem-কে ছোট Subproblem-এ ভাঙতে হয় কীভাবে;

- Single-pass Solution কী;
- Multi-pass Solution কী;
- কখন এক Traversal যথেষ্ট;
- কখন দুইটি Traversal দরকার;
- Pattern Dependency কী;
- Maximum + Counter কীভাবে Combine হয়;
- Condition + Maximum কীভাবে কাজ করে;
- Search + Count কীভাবে Combine হয়;
- Accumulator + Counter দিয়ে Average কীভাবে বের হয়;
- Multiple State Variable কীভাবে Track করতে হয়;
- Problem Statement থেকে Pattern Pipeline কীভাবে তৈরি করতে হয়।

আজকের Core Mental Model:

```
Big Problem
  ↓
Break into Small Questions
  ↓
Identify Pattern for Each Question
  ↓
Check Dependency
  ↓
Combine Patterns
  ↓
Choose One Pass or Multiple Passes
  ↓
Code
```

Lesson 1 — Mixed Pattern Problem কী?

ধরো Problem:

Array-এর Maximum Number কতবার আছে?

Input:

7

5 12 7 12 3 12 8

Expected Output:

3

এটা শুধু Maximum Problem না।

আবার শুধু Counting Problem-ও না।

প্রথমে জানতে হবে:

Maximum Value কী?

তারপর জানতে হবে:

Maximum Value কতবার আছে?

অর্থাৎ:

Problem

↓

Subproblem 1

Find Maximum

↓

Subproblem 2

Count Maximum Occurrences

Pattern:

Traversal
↓
Comparison
↓
Maximum Update

+

Traversal
↓
Target Comparison
↓
Counter

এটাই Mixed Pattern Problem।

Lesson 2 — Problem Decomposition

একটা বড় Problem দেখলে সঙ্গে সঙ্গে Code লিখবে না।

Problem-টাকে প্রশ্নে ভাঙবে।

ধরো:

Find the average of all positive numbers.

প্রথমে প্রশ্ন:

কোন Values দরকার?

Answer:

Positive Values

তাই:

Condition

তারপর:

Average বের করতে কী দরকার?

Formula:

$$\text{Average} = \text{Sum} / \text{Count}$$

তাই দরকার:

Accumulator
+
Counter

সব Values দেখতে হবে:

Traversal

Final Pipeline:

Traversal
↓
Condition
↓
Positive?
↓
Accumulator + Counter
↓
Average

এটাই Problem Decomposition।

Lesson 3 — Pattern Combination Formula

Mixed Problem-এর জন্য একটা Mental Formula মনে রাখো:

What data must I inspect?

↓

Traversal

What values are relevant?

↓

Condition

What result must I remember?

↓

State Variable

How does that state change?

↓

Update Rule

উদাহরণ:

Count negative numbers.

What data?



All Elements



Traversal

Relevant?



Negative



Condition

Remember?



How Many



Counter

Update?



count++

Final:

Traversal



Condition



Counter

আরেকটা:

Find largest even number.

What data?



All Elements



Traversal

Relevant?



Even Numbers



Condition

Remember?



Current Largest Even



Maximum State

Update?



Compare + Update

Final:

Traversal



Condition



Valid Candidate



Comparison



Maximum Update

Lesson 4 — State Variable কী?

এখন পর্যন্ত আমরা অনেক ধরনের Variable ব্যবহার করেছি।

sum
count
max
min
found
index

এগুলো শুধু সাধারণ Variable না।

Problem Solving-এর দৃষ্টিতে এগুলো হলো:

State Variables

অর্থাৎ Traversal চলার সময় Problem-এর গুরুত্বপূর্ণ Information মনে রাখে।

উদাহরণ:

Variable	কী মনে রাখে?
sum	এখন পর্যন্ত Total
count	এখন পর্যন্ত Match Count
max	এখন পর্যন্ত Largest
min	এখন পর্যন্ত Smallest
found	Target পাওয়া গেছে কি না
index	Match-এর Position

Mixed Problem-এ একাধিক State থাকতে পারে।

যেমন:

Positive Numbers-এর Average।

দরকার:

sum
count

Maximum এবং তার Index।

দরকার:

```
max  
max_index
```

Maximum Value কতবার আছে।

দরকার হতে পারে:

```
max  
count
```

Problem Solve করার আগে প্রশ্ন করবে:

Traversal-এর সময় আমাকে কী কী Information মনে রাখতে হবে?

এটাই State Selection।

Lesson 5 — Single Pass কী?

একবার Array Traverse করে Problem Solve করলে তাকে সহজভাবে:

Single-pass Solution

বলতে পারি।

Problem:

Maximum এবং Minimum একসঙ্গে বের করো।

Input:

```
6  
8 3 15 2 10 7
```

আমরা একবার Traverse করেই দুটো Result বের করতে পারি।

```

int max = arr[0];
int min = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }

    if (arr[i] < min)
    {
        min = arr[i];
    }
}

```

Pattern:

```

One Traversal
  ↓
Current Element
  ↙       ↘
Compare Max Compare Min
  ↓         ↓
Update Max  Update Min

```

এখানে:

Number of Passes = 1

Lesson 6 — Multi-pass কী?

একই Array একাধিকবার Traverse করলে:

Multi-pass Solution

ধরো:

Maximum Value কতবার আছে?

Beginner-friendly Solution:

Pass 1

Find Maximum

Pass 2

Count Maximum

Code:

```
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}

int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] == max)
    {
        count++;
    }
}
```

Pipeline:

Pass 1

↓

Traversal

↓

Comparison

↓

Maximum

Pass 2

↓

Traversal

↓

Target Comparison

↓

Counter

এখানে:

Number of Passes = 2

Lesson 7 — কেন দুই Pass দরকার হতে পারে?

Problem:

Maximum কতবার আছে?

Count করার আগে Target জানা দরকার।

কিন্তু Target হলো:

Maximum

যেটা শুরুতে জানা নেই।

তাই Beginner-friendly reasoning:

Maximum Unknown
↓
Find Maximum First
↓
Now Maximum Known
↓
Count It

এটাকে আমরা বলতে পারি:

Pattern Dependency

কারণ:

Counting Step

নির্ভর করছে:

Maximum Finding Step

-এর Result-এর উপর।

Mental Model:

Phase 2 needs Phase 1 result?
↓
Yes
↓
Multi-phase Thinking

Lesson 8 — এক Pass-এ Maximum Count করা যায়?

হ্যাঁ, যায়।

কিন্তু Logic একটু বেশি careful হতে হবে।

ধরো Array:

5 12 7 12 3 12

State:

max
count

Logic:

New Value > max
↓
New Maximum Found
↓
max = value
count = 1

আর:

New Value = max
↓
Same Maximum Again
↓
count++

Code:

```
int max = arr[0];
int count = 1;

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
        count = 1;
    }
    else if (arr[i] == max)
    {
        count++;
    }
}
```

কিন্তু Bootcamp-এর এই Stage-এ দুই-pass Solution বুঝতে পারা বেশি গুরুত্বপূর্ণ।

কারণ:

```
Clear Correct Solution
>
Clever but Confusing Solution
```

Contest-এ আগে Correct Solution।

Optimization পরে।

Lesson 9 — One Pass না Two Pass?

এই Rule অনুসরণ করতে পারো।

যদি Resultগুলো Independent হয়

যেমন:

Maximum

+

Minimum

দুটো একই Element দেখে আলাদাভাবে Update করা যায়।

তাই:

One Pass

যদি Second Result First Result-এর উপর নির্ভর করে

যেমন:

Find Maximum

↓

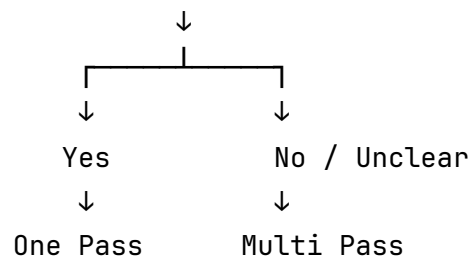
Count Maximum

Beginner-friendly approach:

Two Pass

Mental Rule

Can I update all required states correctly
while seeing each element once?



Lesson 10 — Mixed Problem 1: Count Maximum Occurrences

Problem:

Find how many times the maximum value occurs.

Input:

```
7
5 12 7 12 3 12 8
```

Step 1 — Find Maximum

Pattern:

```
Traversal
↓
Comparison
↓
Maximum Update
```

Result:

```
max = 12
```

Step 2 — Count Maximum

Pattern:

```
Traversal
↓
Target Comparison
↓
Counter
```

Target:

12

Final:

count = 3

Complete Code:

```
#include <stdio.h>

int main()
{
    int n;

    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int max = arr[0];

    for (int i = 1; i < n; i++)
    {
        if (arr[i] > max)
        {
            max = arr[i];
        }
    }

    int count = 0;

    for (int i = 0; i < n; i++)
    {
        if (arr[i] == max)
        {
            count++;
        }
    }

    printf("%d\n", count);

    return 0;
}
```

Lesson 11 — Mixed Problem 2: Average of Positive Numbers

Problem:

Find the average of positive numbers.

Input:

```
6
10 -5 20 -3 30 40
```

Positive Values:

```
10 20 30 40
```

Sum:

```
100
```

Count:

```
4
```

Average:

```
25.00
```

Pattern:

```
Traversal
↓
Condition
↓
Accumulator + Counter
↓
Average
```

Code Logic:

```
int sum = 0;
int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] > 0)
    {
        sum += arr[i];
        count++;
    }
}
```

তারপর:

```
if (count > 0)
{
    double average = (double)sum / count;

    printf("%.2lf\n", average);
}
```

এখানে `count > 0` Check গুরুত্বপূর্ণ।

কারণ:

No Positive Number

↓

count = 0

↓

sum / count

↓

Division by Zero

Lesson 12 — Mixed Problem 3: Largest Even Number

Problem:

Find the largest even number.

Input:

```
7
3 8 5 12 7 20 9
```

Even Values:

```
8 12 20
```

Maximum:

```
20
```

Pattern:

```
Traversal
↓
Condition
↓
Valid Candidate
↓
Comparison
↓
Maximum Update
```

এখানে সরাসরি:

```
int max_even = arr[0];
```

Safe না।

কারণ arr[0] Odd হতে পারে।

Beginner-friendly Solution:

```
int found = 0;
int max_even;

for (int i = 0; i < n; i++)
{
    if (arr[i] % 2 == 0)
    {
        if (found == 0)
        {
            max_even = arr[i];
            found = 1;
        }
        else if (arr[i] > max_even)
        {
            max_even = arr[i];
        }
    }
}
```

Final Decision:

```
if (found == 1)
{
    printf("%d\n", max_even);
}
else
{
    printf("No even number\n");
}
```

এখানে Pattern Combination:

Traversal

+

Condition

+

Flag

+

Maximum

Lesson 13 — Mixed Problem 4: First Positive Number

Problem:

Find the index of the first positive number.

Input:

6

-5 -2 0 8 10 3

Expected:

3

Thinking:

Need every Element?



Traversal

Need specific category?



Condition: `arr[i] > 0`

Need which Match?



First Match

Need what Result?



Index

After First Match?



Break

Final Pattern:

Traversal



Condition



First Valid Match



Save Index



Break

Code:

```
int index = -1;

for (int i = 0; i < n; i++)
{
    if (arr[i] > 0)
    {
        index = i;
        break;
    }
}
```

এখানে আমরা নির্দিষ্ট Target Value খুঁজছি না।

কিন্তু Search Pattern-এর Idea ব্যবহার করছি:

Find First Matching Element

এটা গুরুত্বপূর্ণ।

Search মানেই শুধু:

$arr[i] = target$

না।

Search হতে পারে:

First Positive

First Even

First Number > 100

First Divisible by 7

Lesson 14 — Mixed Problem 5: Last Negative Number

Problem:

Find the index of the last negative number.

Input:

7
5 -2 8 -10 3 -7 9

Negative Indices:

1
3
5

Expected:

5

Thinking:

Traversal
↓
Condition
↓
Match?
↓
Update Index
↓
Continue Search

Code:

```
int index = -1;

for (int i = 0; i < n; i++)
{
    if (arr[i] < 0)
    {
        index = i;
    }
}
```

No break .

কারণ future Element answer বদলাতে পারে।

Lesson 15 — Mixed Problem 6: Sum Between Minimum and Maximum?

এখন Problem Statement পড়ার গুরুত্ব দেখি।

Problem:

Find the sum of the minimum and maximum values.

Input:

5
8 2 15 4 10

Minimum:

2

Maximum:

15

Answer:

17

Pattern:

Traversal

↓

Track Min + Max

↓

min + max

এখানে Element-গুলোর মার্কের Sum করতে হবে না।

Problem-এর ভাষা:

sum of minimum and maximum

মানে:

min + max

এটা:

sum between minimum and maximum

এর থেকে আলাদা Problem।

Contest-এ একটা শব্দের পার্থক্য পুরো Logic বদলে দিতে পারে।

Lesson 16 — Multiple Counters

Problem:

Count Positive, Negative এবং Zero আলাদাভাবে।

Input:

7
5 -2 0 8 -1 0 3

Expected:

Positive: 3

Negative: 2

Zero: 2

State Variables:

positive

negative

zero

Initialization:

```
int positive = 0;
```

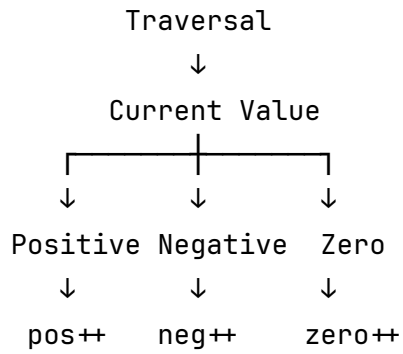
```
int negative = 0;
```

```
int zero = 0;
```

Traversal:

```
for (int i = 0; i < n; i++)  
{  
    if (arr[i] > 0)  
    {  
        positive++;  
    }  
    else if (arr[i] < 0)  
    {  
        negative++;  
    }  
    else  
    {  
        zero++;  
    }  
}
```

Mental Model:



এখানে:

One Traversal
+
Three Counters

Lesson 17 — if vs else if in Classification

আগের Problem:

Positive
Negative
Zero

একটি Number একই সঙ্গে দুই Category-তে পড়তে পারে?

না।

একটি Number হয়:

Positive

অথবা:

Negative

অথবা:

Zero

তাই:

```
if (...)
{
}
else if (...)
{
}
else
{
}
```

LogicalI

এটাকে ভাবতে পারো:

Mutually Exclusive Categories

অর্থাৎ একটির বেশি Category একই সঙ্গে True হবে না।

কিন্তু Problem যদি হয়:

Count numbers divisible by 2 and count numbers divisible by 3.

একটি Number:

6

একই সঙ্গে:

Divisible by 2

এবং:

Divisible by 3

তাই এখানে:

```
if (arr[i] % 2 == 0)
{
    count2++;
}
```

```
if (arr[i] % 3 == 0)
{
    count3++;
}
```

দুইটা independent if লাগতে পারে।

যদি else if দাঁড়:

```
if (arr[i] % 2 == 0)
{
    count2++;
}
else if (arr[i] % 3 == 0)
{
    count3++;
}
```

তাহলে 6 প্রথম Condition-এ ঢোকার পর দ্বিতীয়টা Check হবে না।

এই Bug Contest-এ খুব common।

Lesson 18 — Independent Conditions vs Exclusive Conditions

Exclusive Categories

একটা Value শুধু এক Category-তে যাবে।

Example:

Positive / Negative / Zero

Use:

```
if  
else if  
else
```

Independent Conditions

একটা Value একাধিক Condition Match করতে পারে।

Example:

```
Divisible by 2  
Divisible by 3
```

6 দুইটাই Match করে।

Use:

```
if  
  
if
```

Mental Question:

একটি Element কি একাধিক Category-তে Count হতে পারে?

যদি:

Yes

তাহলে independent Conditions দরকার হতে পারে।

যদি:

No

তাহলে if / else if / else chain উপযুক্ত হতে পারে।

Lesson 19 — Order Matters

Mixed Pattern Problem-এ Operation-এর Order গুরুত্বপূর্ণ হতে পারে।

ধরো Single-pass Maximum Count:

```
if (arr[i] > max)
{
    max = arr[i];
    count = 1;
}
else if (arr[i] == max)
{
    count++;
}
```

কেন নতুন Maximum পেলো:

```
count = 1
```

করছি?

ধরো:

```
5 5 10
```

শুরুতে:

```
max = 5
count = 1
```

দ্বিতীয় 5 :

```
count = 2
```

তারপর 10 :

```
New Maximum
```

এখন আগের 5 -এর Count আর Maximum Count না।

তাই:

```
max = 10  
count = 1
```

অর্থাৎ নতুন Best Result এলে পুরোনো Result-এর সঙ্গে সম্পর্কিত State Reset করতে হতে পারে।

এই Idea ভবিষ্যতে আরও Advanced Problem-এ কাজে লাগবে।

Lesson 20 — Pattern Dependency চিনবে কীভাবে?

Problem:

Maximum Number-এর first index বের করো।

প্রথম চিন্তা হতে পারে:

```
Find Maximum  
↓  
Search First Occurrence
```

এটা দুই Pass।

Phase 1

```
Maximum Pattern
```

Phase 2

First Occurrence Search

Code Logic:

```
int max = arr[0];

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
```

তারপর:

```
int index = -1;

for (int i = 0; i < n; i++)
{
    if (arr[i] == max)
    {
        index = i;
        break;
    }
}
```

Pipeline:

```
Find Maximum
  ↓
Maximum becomes Target
  ↓
Search Target
  ↓
First Match
  ↓
Save Index
```


এখানে Search Phase Maximum Phase-এর উপর নির্ভর করছে।

Lesson 21 — Same Problem, Better Combination

আগের Problem:

Maximum Value এবং তার first index বের করো।

এটা এক Pass-এও করা যায়।

```
int max = arr[0];
int max_index = 0;

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
        max_index = i;
    }
}
```

এখানে:

Maximum Update
+
Index Update

একসঙ্গে হচ্ছে।

কোনটা ব্যবহার করবে?

এই Stage-এ Rule:

যেটা তুমি পরিস্কারভাবে Reason করতে পারো
এবং Correct লিখতে পারো
সেটাই আগে ব্যবহার করো।

Contest-এর প্রথম লক্ষ্য:

Correct

তারপর:

Efficient

তারপর:

Elegant

Lesson 22 — Constraints দেখে Pass Count নিয়ে ভাবা

ধরো:

$n = 100$

একবার Traverse:

100 operations-এর কাছাকাছি

দুইবার Traverse:

200 operations-এর কাছাকাছি

Big-O হিসেবে:

$$\begin{aligned}
 &O(n) + O(n) \\
 &= \\
 &O(2n) \\
 &= \\
 &O(n)
 \end{aligned}$$

তাই দুই Pass মানেই Automatically Bad না।

Beginner হিসেবে:

2 Clear $O(n)$ Passes

অনেক সময়:

1 Confusing $O(n)$ Pass

এর চেয়ে ভালো।

তবে Contest-এ Constraint অবশ্যই দেখবে।

Lesson 23 — Mixed Problem Solving Workflow

এখন থেকে Mixed Problem-এ এই Workflow ব্যবহার করবে।

Step 1 — Output Question

শেষে কী Print করতে হবে?

Step 2 — Required Information

Output বের করতে কী কী Information দরকার?

Step 3 — State Selection

কোন Variables দরকার?

Possible:

sum
count
max
min
found
index

Step 4 — Traversal Requirement

কোন Elements দেখতে হবে?

All?
First Match পর্যন্ত?
Reverse?
Filtered Values?

Step 5 — Dependency Check

এক Result পাওয়ার পর আরেক কাজ শুরু করতে হবে?

যদি Yes:

Multiple Phase / Pass

Step 6 — Update Rules

প্রতিটি State কখন Update হবে?

যেমন:

```
sum
  ↓
sum += value
```

```
count
  ↓
count++
```

```
max
  ↓
if value > max
```

Step 7 — Edge Cases

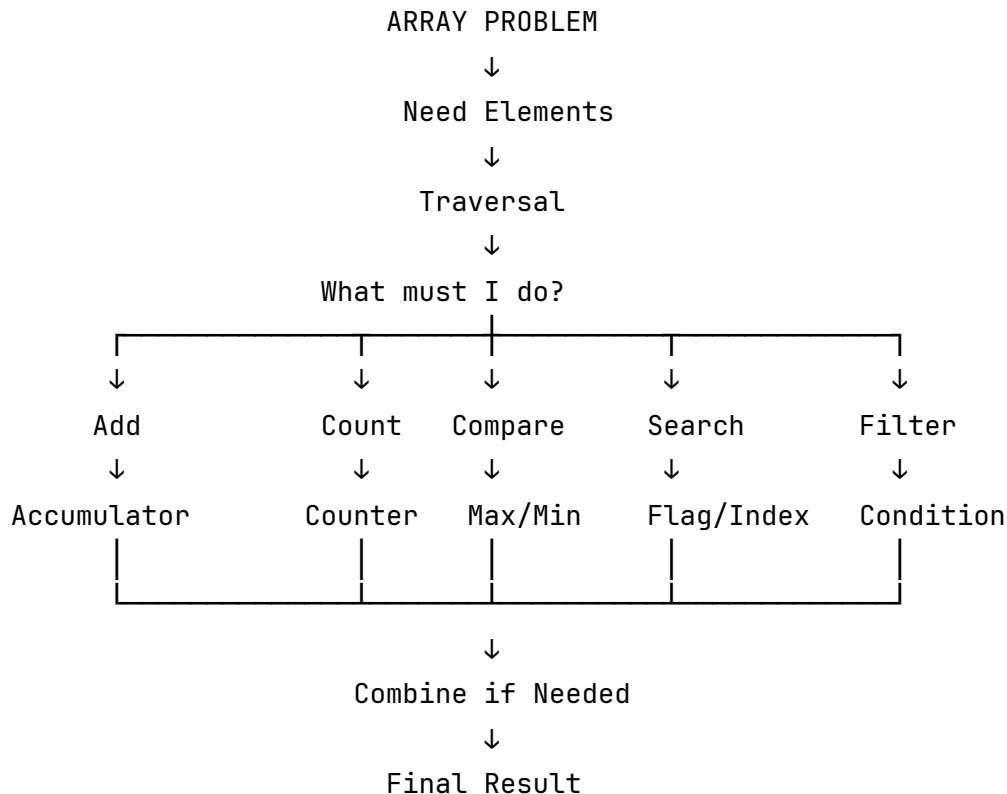
No Match?
All Negative?
All Positive?
One Element?
Duplicates?
Target Missing?

Step 8 — Dry Run

তারপর Code।



Mixed Pattern Master Map



Worked Problem 1 — Count Maximum Occurrences

Problem:

Find how many times the largest number appears.

Input:

8

5 12 7 12 3 12 8 12

Problem Decomposition

Need Count of Maximum

↓

But Maximum Unknown

↓

Find Maximum

↓

Use Maximum as Target

↓

Count Target

Pattern:

Phase 1

Traversal

↓

Comparison

↓

Maximum Update

Phase 2

Traversal

↓

Target Comparison

↓

Counter

Dry Run Phase 1:

Maximum = 12

Phase 2:

12 found 4 times

Answer:

4



Worked Problem 2 — Sum and Count of Even Numbers

Problem:

Find the Sum and Count of Even Numbers.

Input:

7
3 8 5 12 7 10 4

Even:

8 12 10 4

Sum:

34

Count:

4

Pattern:

Traversal

↓

Condition

↓

Even?

↓

Accumulator + Counter

Code Logic:

```
int sum = 0;
int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] % 2 == 0)
    {
        sum += arr[i];
        count++;
    }
}
```

এখানে দুইটা Result Independentভাবে একই Match-এর উপর Update হচ্ছে।

তাই:

One Pass

যথেষ্ট।



Worked Problem 3 — First Maximum Index

Problem:

Find the first index of the maximum value.

Input:

7
5 12 7 12 3 8 12

Expected:

1

Beginner-friendly Decomposition:

Phase 1
Find Maximum
↓
max = 12

Phase 2
Search 12
↓
First Match
↓
Index = 1
↓
Break

Pattern:

Maximum Pattern
↓
Target Search
↓
First Occurrence

Chapter 6 Exercise

এখানে **৮টি Problem** থাকবে। কিন্তু সবগুলোর Full Code একসঙ্গে লিখতে হবে না।

Day 2 একদিনে শেষ করার জন্য ভাগ:

Must Solve Today:

Exercise 1-5

Challenge:

Exercise 6-8

Exercise 1 — Sum and Count Positive Numbers

Input:

```
7
10 -5 20 0 -3 30 5
```

Expected:

```
Sum: 65
Count: 4
```

তুমি লিখবে:

```
Pattern:
???
```

↓

```
???
```

↓

```
???
```

Variables:

```
???
```

```
???
```

তারপর Code।

Exercise 2 — Maximum and Minimum Together

Input:

```
7
8 3 15 2 10 1 7
```

Expected:

```
Maximum: 15
Minimum: 1
```

Requirement:

```
One Traversal
```

Exercise 3 — Count Maximum Occurrences

Input:

```
8
5 12 7 12 3 12 8 12
```

Expected:

```
4
```

Requirement:

```
Beginner-friendly Two Pass Solution
```

প্রথমে লিখবে:

Phase 1:

...

Phase 2:

...

তারপর Code।

Exercise 4 — First Positive Index

Input:

6
-5 -2 0 8 10 3

Expected:

3

নিজে ঠিক করবে:

Flag দরকার?

Index দরকার?

break দরকার?

Exercise 5 — Count Positive, Negative and Zero

Input:

8
5 -2 0 8 -1 0 3 -7

Expected:

Positive: 3

Negative: 3

Zero: 2

Requirement:

One Traversal

+

Three Counters

Exercise 6 — Challenge: Largest Odd Number

Input:

7

8 3 12 7 20 15 4

Expected:

15

এই Test Case-টাও করবে:

5

2 4 6 8 10

এখানে কোনো Odd Number নেই।

Pattern:

Traversal
↓
Condition
↓
???
↓
Comparison
↓
Update

Exercise 7 — Challenge: First and Last Occurrence

Input:

```
9
5 2 7 2 9 2 10 2 8
2
```

Expected:

```
First: 1
Last: 7
```

Requirement:

```
One Full Traversal
```

States:

```
first
last
```

নিজে Logic তৈরি করবে।

Exercise 8 — Challenge: Count Minimum Occurrences

Input:

```
8
5 2 7 2 9 2 10 3
```

Minimum:

```
2
```

Count:

```
3
```

Beginner-friendly Solution:

```
Phase 1:
???
```

```
Phase 2:
???
```



Chapter 6 Assignment

এই Assignment-এর মূল লক্ষ্য Code না।

মূল লক্ষ্য:

Problem ভেঙে Pattern চিনতে পারা।

Task 1 — Pattern Breakdown

Problem:

Find the average of all even numbers.

লিখবে:

Required Information:

- 1. ???
- 2. ???

Pattern:

???
↓
???
↓
???
↓
???

Variables:

???
???

Task 2 — Dependency Analysis

Problem:

Count how many times the minimum value occurs.

লিখবে:

What must be known first?

...

Phase 1:

...

Phase 2:

...

Complete Pattern:

...

Task 3 — State Selection

Problem:

Find Maximum Value and its first occurrence index.

লিখবে:

State Variables:

1. ???
2. ???

When should each state update?

...

Should equal maximum update the index?

Yes / No

Why?

...

Task 4 — Condition Logic

Problem:

Count numbers divisible by 2 and count numbers divisible by 3.

Input:

6
2 3 6 8 9 12

প্রশ্ন:

Should I use:

if + if

or

if + else if

কারণ লিখবে।

তারপর নিজে Expected Count বের করবে।

Task 5 — Pattern Combination

Problem:

Find the largest negative number.

Example:

5 -2 7 -10 -3

Answer:

-2

লিখবে:

Pattern:

???
↓
???
↓
???
↓
???

Initialization Challenge:

...

No Negative Number থাকলে:

...

খেয়াল করো: Chapter 4-এর Assignment-এ ছিল *smallest negative number*, আর এখানে *largest negative number*। $-2 > -10$, তাই Answer -2 ।

Task 6 — Bug Hunt

Problem:

Count numbers divisible by both 2 and 3.

Code:

```
int count = 0;

for (int i = 0; i < n; i++)
{
    if (arr[i] % 2 == 0)
    {
        count++;
    }

    if (arr[i] % 3 == 0)
    {
        count++;
    }
}
```

Answer করবে:

Bug:

...

Why?

...

Correct Condition:

...

Hint:

Divisible by 2

AND

Divisible by 3

একই Number-এর জন্য দুইটা আলাদা Count না।

Task 7 — Design the Pipeline

Problem:

Find the first occurrence of the minimum value.

শুধু Pipeline লিখবে।

Format:

Phase 1

???

↓

???

↓

???

Phase 2

???

↓

???

↓

???

↓

???

তারপর ব্যাখ্যা করবে কেন Phase 2, Phase 1-এর Result-এর উপর নির্ভর করছে।



Chapter 6 Common Mistakes

Mixed Problem-এ সবচেয়ে বেশি যে ভুলগুলো হয়:

Mistake	Problem
Code আগে লেখা	Pattern পরিষ্কার না
State ভুল নেওয়া	Result মনে রাখা যায় না
break ভুল জায়গায়	Search অসম্পূর্ণ
if বনাম else if ভুল	Count ভুল
Invalid initialization	Max/Min ভুল
Dependency না বোঝা	Wrong phase order
Edge case ignore	Hidden test fail
একই Variable ভুল কাজে ব্যবহার	State corrupt



Chapter 6 Quick Revision Sheet

Simple Sum

Traversal

↓

Accumulator

Conditional Sum

Traversal
↓
Condition
↓
Accumulator

Conditional Count

Traversal
↓
Condition
↓
Counter

Maximum

Traversal
↓
Comparison
↓
Update

Conditional Maximum

Traversal
↓
Condition
↓
Valid Candidate
↓
Comparison
↓
Update

Existence Search

```
Traversal  
↓  
Target Comparison  
↓  
Flag  
↓  
Break
```

First Match

```
Traversal  
↓  
Condition / Target Comparison  
↓  
Save Index  
↓  
Break
```

Last Match

```
Traversal  
↓  
Condition / Target Comparison  
↓  
Update Index  
↓  
Continue
```

Average of Matching Values

Traversal

↓

Condition

↓

Accumulator + Counter

↓

Division

Count Maximum

Find Maximum

↓

Use Maximum as Target

↓

Traversal

↓

Counter



The Most Important Skill of Chapter 6

একটা Problem দেখে:

এইটা Maximum Problem

এতটুকু বললে এখন আর যথেষ্ট না।

তোমাকে পুরো Pipeline বলতে হবে।

যেমন:

Count how many times the maximum appears.

তোমার Brain:

Maximum আগে জানা দরকার



Phase 1

Traversal



Comparison



Maximum Update



Maximum এখন Target



Phase 2

Traversal



Target Comparison



Counter

আবার:

Find first positive number.

Brain:

Traversal



Condition



First Valid Match



Save Index

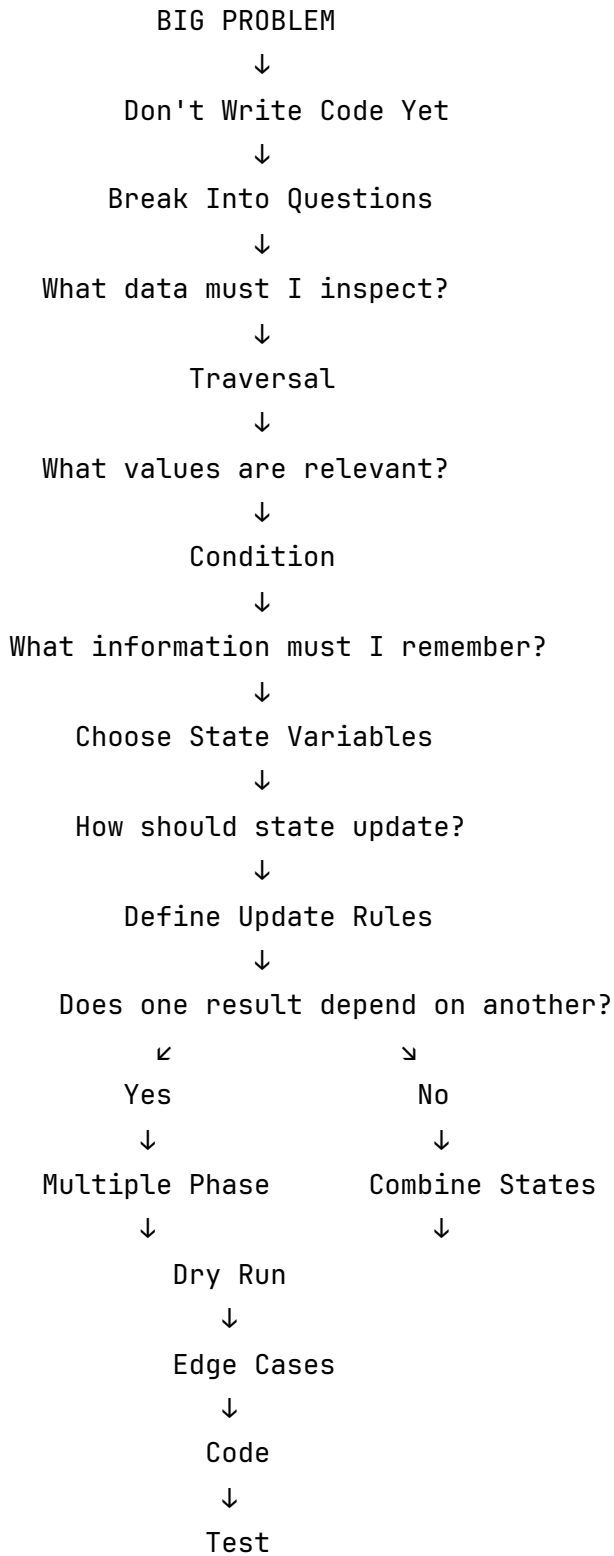


Break

এইভাবে **Problem** → **Pipeline** translate করতে পারা CP-এর মূল skillগুলোর একটি।



Chapter 6 Final Mental Model





Day 2 Progress

Day 2

- ✓ Chapter 1 – Array Mental Model
- ✓ Chapter 2 – Traversal Pattern
- ✓ Chapter 3 – Accumulation + Counting
- ✓ Chapter 4 – Maximum + Minimum
- ✓ Chapter 5 – Linear Search
- ✓ Chapter 6 – Mixed Pattern Problems
- ⌚ Chapter 7 – Mistake Review, Pattern Library & Day 2 Final Assignment

Chapter 6 পর্যন্ত Day 2-এর **সব Core Problem-Solving Pattern শেখা শেষ**। Chapter 7-এ আর বড় নতুন Theory যোগ হবে না। সেখানে Day 2-এর common mistakes, edge-case checklist, debugging workflow, `core_pattern.md` update, final mixed assignment এবং Day 2 completion checklist থাকবে—যাতে Day 2 সত্যিই একদিনের মধ্যে close করে Day 3-তে যাওয়া যায়।